LLM-BASED WHEEL HUMAN INTERACTION ROBOT FOR RECEPTIONIST AND HUMAN ASSISTANT

KIRAWUT CHALERMKITPAISAN
KRITTIN SAKHARIN
NATCHA RUNGRUANG
NATWASA MANOMAIWIBOON

A THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENT FOR THE DEGREE OF
BACHELOR OF ENGINEERING IN ROBOTICS AND AI ENGINEERING
SCHOOL OF ENGINEERING
KING MONGKUT'S INSTITUTE OF TECHNOLOGY LADKRABANG
2025

LLM-BASED WHEEL HUMAN INTERACTION ROBOT FOR RECEPTIONIST AND HUMAN ASSISTANT

KIRAWUT CHALERMKITPAISAN
KRITTIN SAKHARIN
NATCHA RUNGRUANG
NATWASA MANOMAIWIBOON

A THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENT FOR THE DEGREE OF
BACHELOR OF ENGINEERING IN ROBOTICS AND AI ENGINEERING
SCHOOL OF ENGINEERING
KING MONGKUT'S INSTITUTE OF TECHNOLOGY LADKRABANG
2025

Thesis	LLM-based Wheel Human Interaction Robot for Receptionist and Human Assistant
Student	Mr. Kirawut Chalermkitpaisan, Mr. Krittin Sakharin,
	Ms. Natcha Rungruang, Ms. Natwasa Manomaiwiboon
Student ID.	65011333, 65011356, 65011386, 65011402
Degree	Bachelor of Engineering
Program	Robotics and AI Engineering
Year	2025
Thesis Advisor	Asst. Prof. Dr. Sarucha Yanyong

ABSTRACT IN ENGLISH

This thesis presents the development of Satu, a wheeled robotic assistant created within the Robotics and AI Engineering program at King Mongkut's Institute of Technology Ladkrabang. The primary objective was to build an accessible robot capable of assisting with academic tasks, such as answering department-related questions in Thai and guiding users to specific locations within Building E-12. To achieve this, the system integrates a Large Language Model (LLM) supported by a knowledge retrieval mechanism for contextual conversation, while autonomous navigation is handled via a ROS2-based framework and pre-constructed floor maps.

The unified system coordinates speech processing, visual perception, and navigation across multiple embedded computing devices, all housed within a physical design optimized for a welcoming user experience. Evaluation through iterative testing showed consistent improvements in conversation understanding and navigation accuracy, with the final version meeting all predefined performance criteria. These findings suggest that the integration of conversational AI and mobile robotics provides a reliable and functional solution for real-world academic environments.

ACKNOWLEDGEMENT

We would like to express our sincere appreciation to Asst. Prof. Dr. Sarucha Yanyong, thesis advisor at the Department of Robotics and AI Engineering, School of Engineering, King Mongkut's Institute of Technology Ladkrabang, for his guidance, insightful feedback, and continuous support throughout this project. His knowledge and dedication greatly influenced the direction and overall quality of this work.

We also extend our thanks to the professors, faculty members, and staff of the Department of Robotics and AI Engineering at KMITL for providing access to laboratory facilities, technical resources, and a supportive research environment. Their support in resolving technical challenges at various stages of the project is greatly appreciated.

In addition, we are grateful to our friends in the RAI program who participated in system testing and evaluation. Their interaction with the prototype, along with their valuable feedback, was essential in assessing the system's performance and reliability under realistic conditions. Their guidance, encouragement, and collaboration throughout the project made this experience both productive and enjoyable.

Finally, we are deeply grateful to our families for their constant encouragement and understanding, which gave us the strength to persevere through challenges and complete this thesis.

Kirawut Chalermkitpaisan, Krittin Sakharin,

Natcha Rungruang, and Natwasa Manomaiwiboon

TABLE OF CONTENTS

Page

LIST OF TABLES

Table **Page**

LIST OF FIGURES

Figure **Page**

LIST OF ABBREVIATIONS AND SYMBOLS

T	Torque
F	Force applied
F_r	Frictional force
F_i	Inertial force
r	Radius
m	Mass of the object
a	Acceleration of the object
g	Acceleration due to gravity
l	Length
v	Velocity
ω	Angular velocity
d	Distance
V	Voltage
I	Current
P	Power
D	Duty cycle
η	Efficiency
E	Energy
C	Capacitance
T	Time period

CHAPTER 1

INTRODUCTION

1.1 STATEMENT AND SIGNIFICANCE OF THE PROBLEM

In modern educational institutions, students require timely access to information and assistance in navigating increasingly complex campus environments. Conventional support systems, such as static information boards and staffed reception desks, are often limited in availability and lack the flexibility needed to address diverse and dynamic user needs efficiently. As a result, students may experience delays in obtaining information or difficulties in navigating unfamiliar locations. This limitation highlights the need for a more responsive, intelligent, and continuously available support system.

Recent advancements in artificial intelligence and robotics have enabled the development of interactive robotic assistants capable of addressing these challenges. Such systems can provide real-time navigation assistance, respond to a wide range of user queries, and enhance the overall accessibility and engagement of campus services. When integrated with appropriate AI technologies, robotic assistants can operate autonomously, adapt to environmental changes, and deliver consistent and accurate information on demand.

This thesis proposes the development of an intelligent robotic assistant designed for deployment within a controlled campus environment. The system integrates Simultaneous Localization and Mapping (SLAM) for autonomous indoor navigation, voice recognition technologies for natural hands-free interaction, and a Retrieval-Augmented Generation (RAG) framework to deliver accurate, context-aware responses based on domain-specific knowledge. Collectively, these components form a unified system intended to provide an efficient and intelligent support solution for students within the E12 Building at KMITL.

1.2 GOAL AND OBJECTIVE OF THE STUDY

- To design and integrate a mobile robotic platform equipped with LiDAR sensors for real-time obstacle detection and SLAM-based autonomous navigation within the E12 Building.
- To incorporate a Large Language Model (LLM) to enable natural, context-aware conversations between the robot and users.
- To implement a Retrieval-Augmented Generation (RAG) pipeline to enhance response accuracy using curriculum-specific and domain-relevant knowledge.
- To develop a hybrid data management system utilizing MySQL for structured data storage and Milvus for vector-based semantic retrieval.
- To integrate Speech-to-Text (STT) and Text-to-Speech (TTS) systems to support fully hands-free voice interaction.
- To design a user-friendly interface that is accessible and intuitive for users with varying levels of technological proficiency.
- To optimize the overall system for real-time performance, ensuring minimal latency in navigation, speech processing, and response generation.

1.3 HYPOTHESIS

1.3.1 Autonomous Navigation and Obstacle Avoidance

The integration of LiDAR sensors and SLAM algorithms enables the mobile robot to achieve accurate real-time localization and effective obstacle avoidance within the E12 Building environment. This capability is supported by the principles of probabilistic robotics and Simultaneous Localization and Mapping (SLAM), which allow continuous map construction and updating in dynamic environments.

1.3.2 Effectiveness of Natural Language Interaction

The incorporation of a Large Language Model (LLM) significantly enhances the system's ability to engage in natural, context-aware interactions with users. This is supported by advancements in transformer-based architectures, which demonstrate

strong performance in understanding and generating human-like language.

1.3.3 Improved Response Accuracy through Retrieval-Augmented Generation (RAG)

The integration of a Retrieval-Augmented Generation (RAG) pipeline improves the factual accuracy and relevance of generated responses compared to a standalone language model. This is supported by information retrieval theory, where external knowledge sources enhance response grounding and reduce hallucination.

1.3.4 Efficiency of Data Management System

The combination of MySQL for structured data storage and Milvus as a vector database enables efficient storage, retrieval, and management of both interaction logs and knowledge representations. This approach is grounded in database system principles and vector similarity search theory.

1.3.5 Usability of Voice-Based Interaction

The integration of Speech-to-Text (STT) and Text-to-Speech (TTS) systems improves accessibility and interaction efficiency by enabling hands-free communication. This aligns with human–computer interaction (HCI) principles that emphasize natural and intuitive interfaces.

1.3.6 User Interface Accessibility and Acceptance

A user-friendly interface enhances usability and user satisfaction across users with varying levels of technological familiarity. This is supported by usability engineering and user-centered design principles.

1.3.7 Real-Time System Performance

Optimizing the integration of navigation, AI processing, and speech pipelines results in low-latency system performance, enabling seamless real-time interaction. This is supported by real-time systems theory, where responsiveness is critical for effective human–robot interaction.

1.4 SCOPE OF THE STUDY

This study focuses on the development and evaluation of the proposed robotic assistant within a defined operational environment. The system is deployed exclusively on the 12th floor of the E12 Building, which serves as the primary testing and demonstration area. All mapping, navigation configuration, and system evaluations are conducted within this environment, and the system is not intended for operation beyond this.

The primary target users are students in the Robotics and Artificial Intelligence (RAI) programme. The system is designed to address use cases relevant to this group, including indoor navigation assistance and conversational interaction. The robot supports two main functionalities: providing directions to designated locations and engaging in general dialogue through an LLM-based interaction system.

1.5 PROCESS OF THE STUDY

The development of the robotic assistant follows a structured methodology consisting of five sequential phases, from mechanical design to system evaluation.

1.5.1 Robot Mechanical Design

This phase involves the design of the robot's physical structure. The chassis is configured to accommodate all necessary components, including computing units, sensors, batteries, and drive mechanisms, while ensuring structural stability and balanced weight distribution.

1.5.2 Motor Selection and Placement

This phase focuses on selecting appropriate motors based on payload requirements, desired speed, and maneuverability. Motor placement is optimized for efficient movement, and controllers are integrated using the ROS 2 framework.

1.5.3 Control System Design

The control system architecture is developed by integrating the ROS 2 navigation stack, LiDAR-based SLAM for mapping and localization, and Adaptive Monte Carlo

Localization (AMCL) for real-time pose estimation. Communication between system components is managed through ROS 2 topics and services.

1.5.4 Navigation and Interaction

This phase integrates navigation and interaction modules. The conversational system combines an LLM with a RAG pipeline supported by a vector database. STT and TTS systems are incorporated for voice interaction, and a face detection module enables proactive engagement with users.

1.5.5 Testing and Evaluation

System testing and evaluation are conducted within the operational environment. Performance metrics include navigation accuracy, speech recognition quality, response relevance, and user satisfaction, evaluated through controlled experiments with RAI students.

1.6 ASSUMPTIONS

1.6.1 Hardware Capability

The onboard computing system is assumed to be capable of handling real-time processing tasks, including navigation, speech processing, and AI inference.

1.6.2 Power Availability

The robot’s power system is assumed to provide sufficient energy for continuous operation throughout testing periods.

1.6.3 Environmental Conditions

The operational environment is assumed to be semi-structured, with minimal unpredictable changes. Static elements such as walls and corridors remain stable, supporting reliable SLAM performance.

1.6.4 Network Connectivity

A stable network connection is assumed to be available when required, particularly for accessing cloud-based services and external knowledge sources.

1.7 LIMITATION OF THE STUDY

- The system may require periodic updates to improve interaction accuracy and maintain performance.
- Budget constraints may limit the use of high-end hardware components.
- Environmental factors such as lighting conditions, noise levels, and physical obstacles may affect system performance.

1.7.1 Technical Constraints

- Hardware Limitations
  - Computational Power: The onboard system must efficiently handle AI processing without excessive heat or power consumption.
  - Battery Life: Energy efficiency is required to maximize operational duration.
  - Mechanical Design: The robot must ensure stable and smooth movement.
  - Sensor Accuracy: Sensors must provide reliable input while balancing cost and power constraints.
  - Weight Distribution: Proper balance is required to maintain stability.
- Software and AI Limitations
  - Real-Time Processing: Latency in speech or navigation systems may affect performance.
  - LLM Integration: Requires efficient inference and contextual accuracy.
  - Navigation and Mapping: SLAM performance depends on environmental conditions.
  - Robustness: The system must handle variations in lighting, noise, and environment.

1.7.2 Non-Technical Design Constraints

- User Acceptance and Adaptability

  The system must be intuitive and accessible to users with varying levels of technical expertise.

- Privacy and Data Security

  User data must be handled in accordance with data protection and privacy standards.

- Cost and Scalability

  The system should balance affordability with performance to support broader deployment.

- Regulatory and Ethical Considerations

  The system must comply with safety standards and ethical AI practices, ensuring fairness, transparency, and responsible deployment.

1.8 DEFINITIONS

Table 1.1 Definitions of Key Terms and Abbreviations

Abbreviation	Full Term	Definition
FDM	Fused Deposition Modeling	3D printing technique that builds objects layer by layer using thermoplastic materials.
PLA	Polylactic Acid	Biodegradable thermoplastic commonly used in FDM 3D printing.
AMCL	Adaptive Monte Carlo Localization	Probabilistic localization algorithm for ROS-based autonomous navigation.
ASR	Automatic Speech Recognition	Technology that converts spoken speech into written text.
LLM	Large Language Model	AI model trained on large text datasets for understanding and generating human language.
NLP	Natural Language Processing	Subfield of AI concerned with enabling computers to understand human language.
RAG	Retrieval-Augmented Generation	Technique that enhances LLM outputs by retrieving relevant external information.
ROS	Robot Operating System	Open-source middleware framework for developing robot software.
STT	Speech-to-Text	Process of converting spoken audio signals into written text.
TTS	Text-to-Speech	Speech synthesis technology that converts written text into audible output.

CHAPTER 2

LITERATURE REVIEW

2.1 MECHANICAL DESIGN

This section covers the mechanical aspects of the robotic assistant's design, including chassis, mobility systems, and structural components.

2.1.1 Structural and Exterior Design

The structural and exterior design of a service robot should support both mechanical integration and user perception. Structurally, the robot body must provide space for major components such as sensors, batteries, controllers, and support members. At the same time, the visible body proportion influences how the robot is interpreted by users. Trovato et al. showed that variations in chest-to-hip ratio (CHR) and waist-to-hip ratio (WHR) affect the visual perception of robot bodies, indicating that torso proportion is an important aspect of robot embodiment design [1], [2].

In service robots, the body is commonly divided into functional sections such as the base and the upper body. This arrangement helps separate locomotion hardware from interaction-related components and improves internal organization. The cited study also suggests that body contour and proportion can be used deliberately as design cues, especially in the torso region, where changes in the waist and hip relationship influence the overall visual impression of the robot. As shown in Figure 2.1, the study presented six robot body forms created from different combinations of CHR and WHR.

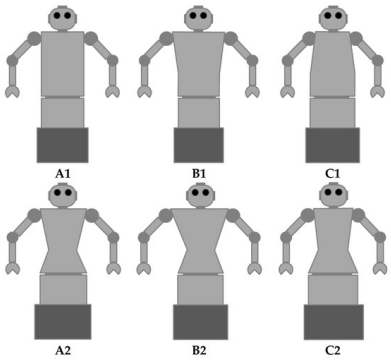


Figure 2.1 Six robot body forms created from different combinations of chest-to-hip ratio (CHR) and waist-to-hip ratio (WHR) [3].

Therefore, structural and exterior design should be considered together, since robot body proportion and torso contour influence not only hardware integration but also the visual impression of the robot. Figure 2.2 presents the perception results reported for the different body types, showing that changes in body proportion can lead to different interpretation tendencies [3].

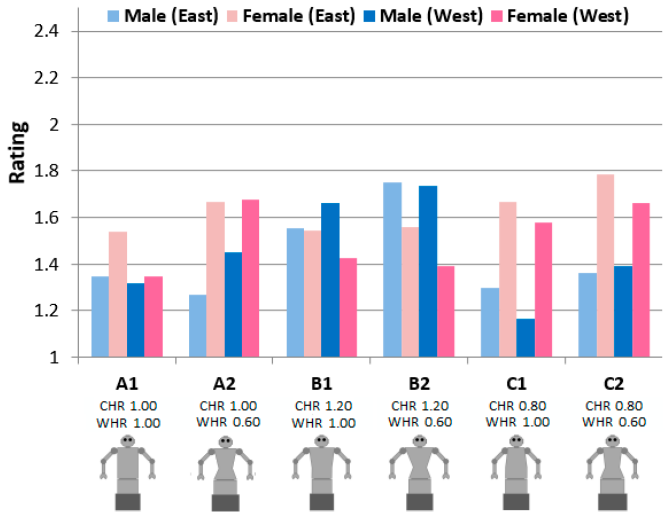


Figure 2.2 Comparison of declared gender perception ("he"/"she") for the six robot body stimuli between Eastern and Western participant groups [3]

2.1.2 Material Selection

Material selection is an important part of robot mechanical design because it affects structural support, total weight, manufacturability, electrical safety, and overall cost. In service robots, the selected materials must provide sufficient rigidity for internal components while also being suitable for the required exterior shape and fabrication process. For this reason, robot structures often use more than one material, with each material selected according to its functional role.

2.1.2.1 Aluminum

Aluminum is commonly used as a structural material in robots because it provides a good balance between strength and low weight. It is suitable for frames, support members, and mounting structures that must carry mechanical and electrical components. In addition, aluminum can be machined and assembled conveniently, making it practical for modular robot construction.

Table 2.1 Material Properties and Advantages

Material	Main Properties	Advantages
Aluminum	Lightweight, high strength-to-weight ratio, corrosion resistance, machinability	Good strength-to-weight ratio, easy to machine and assemble
Acrylic	Lightweight, easy to fabricate, electrical insulation properties	Easy to cut, suitable for insulation and simple support panels
PLA	Lightweight thermoplastic, suitable for FDM printing, low cost	Easy to print, suitable for curved and customized shapes

2.1.2.2 Acrylic

Acrylic is often used in robot assemblies where a flat supporting surface and electrical insulation are required. It is lightweight, easy to fabricate, and suitable for non-primary structural parts such as electrical support panels and trays. Although acrylic is not usually used as the main load-bearing structure, it is useful in sections where insulation and simple fabrication are more important than high structural strength.

2.1.2.3 PLA and Additive Manufacturing

PLA is commonly selected for prototype development because it is lightweight and suitable for FDM 3D printing [4]. This makes it appropriate for robot body panels, covers, and custom housings, especially when the design requires smooth and customized external geometry. However, PLA is not ideal for load-bearing structures or components that require high durability, as it can be brittle and less resistant to mechanical stress compared to materials like aluminum.

2.1.3 Torque

Torque is a measure of the rotational force applied to an object, calculated as the product of the force and the distance from the pivot point. In robotics, torque is crucial for determining the robot's ability to perform tasks that require rotation or movement against resistance. The required torque for a robotic joint can be calculated

using the formula:

$$T = Fxr \tag{2.1}$$

- Torque Calculation for Robot Arm Motors For robotic arms, torque is required to lift and manipulate objects. The torque needed is influenced by the mass of the object, the length of the arm, and gravitational force. The torque can be calculated using the formula:

$$T = mxgxl \tag{2.2}$$

If the arm consists of multiple segments, the total torque required is the sum of the torques from each segment:

$$T_{total} = \sum_{i=1}^{n} (m_i x g x l_i) \tag{2.3}$$

- Torque Calculation for Mobile Robot Wheels For mobile robots, torque is required to overcome friction and move the robot. The torque needed can be calculated using the formula:

$$T = (M \times a \times F_r \times F_i) \times r \tag{2.4}$$

Since frictional losses and real-world conditions can impact performance, the actual motor torque should be selected with a safety margin, typically 1.5 to 2 times the calculated torque.

2.1.4 Kinematics

Kinematics is the study of motion without consideration of the forces that cause it. In mobile robotics, kinematics describes the geometric relationship between actuator commands and the resulting motion of the robot in its environment. Understanding robot kinematics is essential for trajectory planning, odometry-based position estimation, and the design of motion control systems.

The robot employed in this project uses a differential drive configuration, in which two wheels are independently actuated on a shared axle. This configuration is widely adopted in indoor mobile robots due to its mechanical simplicity and ease of control. The instantaneous linear velocity v and angular velocity ω of a differential drive robot are derived from the individual wheel velocities as follows:

$$v = \frac{v_R + v_L}{2} \tag{2.5}$$

$$\omega = \frac{v_R - v_L}{d} \tag{2.6}$$

The robot's pose in a two-dimensional plane is described by the state (x, y, θ), where (x, y) denotes the position and θ is the heading angle. The body-frame velocity components and heading angle used to describe the robot motion are illustrated in Figure X.

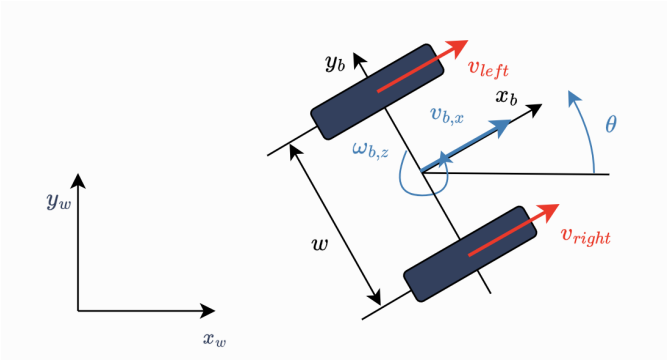


Figure 2.3 Differential Drive Robot Diagram

The time derivatives of the pose components are given by:

$$\dot{x} = v\cos\theta, \quad \dot{y} = v\sin\theta, \quad \dot{\theta} = \omega \tag{2.7}$$

In practice, the robot's pose is continuously estimated through odometry by integrating these kinematic equations using wheel encoder measurements. However, integration over time accumulates errors introduced by wheel slip, encoder resolution limitations, and surface irregularities, resulting in odometry drift. To compensate for this drift, localisation algorithms such as Adaptive Monte Carlo Localisation (AMCL) fuse odometry data with external sensor observations, such as LiDAR scan matching, to maintain an accurate and consistent pose estimate during autonomous operation.

2.2 ELECTRICAL DESIGN

This section covers the electrical systems, power management, and hardware integration aspects of the robot.

2.2.1 Power Supply Architecture and Electrical Safety

The power supply architecture of a mobile robotic system determines how electrical energy is arranged, distributed, and protected throughout the platform. In addition to supplying the required voltage and current to each subsystem, the electrical design must also ensure safe operation under normal and fault conditions. For this reason, both connection configuration and protection devices are important considerations in robotic electrical systems.

In electrical design, the arrangement of components can be classified into series and parallel connections. These two connection types have different electrical characteristics and are used for different purposes in robotic systems.

2.2.1.1 Series and Parallel Connection

A series connection is formed when components are connected in a single continuous path, causing the same current to flow through every element. In this arrangement, the total voltage across the circuit is equal to the sum of the voltage drops across each component to increase the voltage. The total voltage of a series-

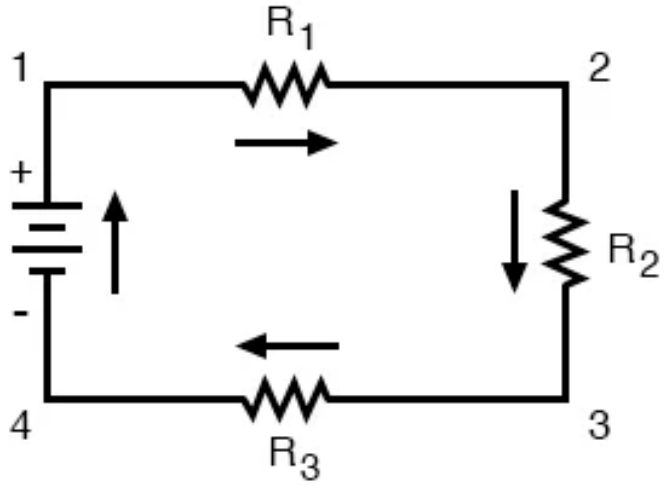


Figure 2.4 Series circuit diagram

connected source can be expressed as:

$$V_{total} = \sum_{i=1}^{n} (V_i) \tag{2.8}$$

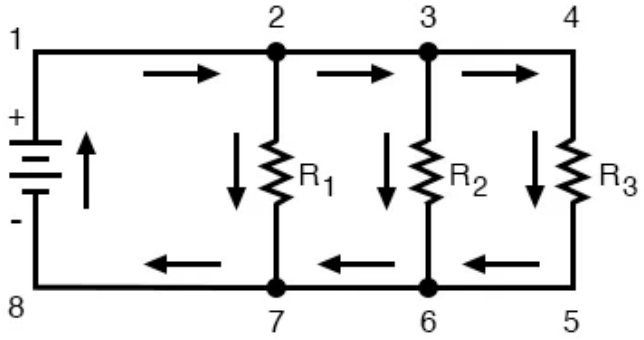


Figure 2.5 Parallel circuit diagram

A parallel connection provides multiple paths for current flow. In this arrangement, each branch receives the same supply voltage, while the total current is shared among the loads according to their individual requirements. The total current in a parallel circuit is given by:

$$I_{total} = \sum_{i=1}^{n} (I_i) \tag{2.9}$$

2.2.1.2 Electrical Safety Considerations

Electrical safety considerations are important in robotic systems to prevent damage caused by reverse polarity, overcurrent, electrical transients, and hazardous operat-

ing conditions. Diodes are commonly used for reverse polarity protection and flyback suppression in inductive circuits. Circuit breakers are used to disconnect the power path when abnormal current is detected, protecting the system from overheating and electrical faults. In addition, an emergency stop (EMO) provides a manual method for immediately shutting down robot operation during unsafe situations. Proper wiring, grounding, and component selection further contribute to the safe and reliable operation of the robotic platform.

2.2.2 DC to DC Buck Converter

A DC to DC buck converter is a type of switching voltage regulator used to reduce a higher direct current (DC) input voltage to a lower regulated DC output voltage [5]. It is widely used in embedded systems, portable electronics, and mobile robotic platforms because many electronic components operate at voltage levels lower than the main battery supply. In robotic applications, a buck converter is especially important for powering low-voltage devices such as microcontrollers, sensors, single-board computers, and communication modules from a higher-voltage battery source.

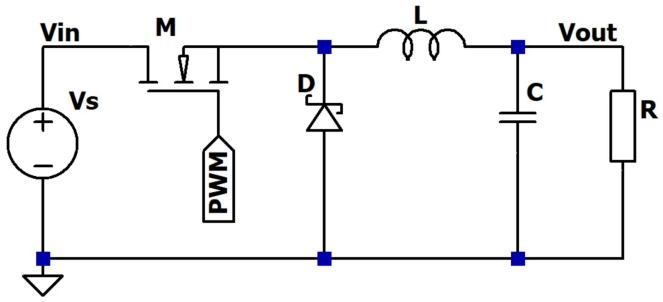


Figure 2.6 Buck Converter Circuit Diagram [5]

The operation of a buck converter is based on the controlled switching of a semiconductor device, typically a transistor or MOSFET, together with an inductor, diode or synchronous switch, and output capacitor. When the switch is turned on, energy from the input source is transferred to the inductor and load. During this period, the inductor stores energy in its magnetic field. When the switch is turned off, the inductor releases its stored energy to the load through the freewheeling path, allowing the output current to continue flowing. The capacitor helps smooth the output voltage by reducing voltage ripple caused by the switching action. The output voltage of an ideal

buck converter can be expressed as:

$$V_{out} = D \times V_{in} \tag{2.10}$$

In practical circuits, the output voltage is also influenced by component losses, switching frequency, load current, and internal resistance of the converter elements. Therefore, real buck converters are designed with feedback control mechanisms to maintain a stable output voltage even when the input voltage or load conditions vary. This voltage regulation characteristic is essential in robotic systems, where electronic modules may be sensitive to voltage fluctuations.

The efficiency of a converter is defined as the ratio of output power to input power, expressed as a percentage:

$$\eta = \left(\frac{P_{out}}{P_{in}}\right) x100\% \tag{2.11}$$

In summary, the DC to DC buck converter is a key component in modern robotic electrical systems because it provides efficient and regulated voltage conversion from the main battery supply to lower-voltage electronic subsystems. Its high efficiency, controllable output, and suitability for battery-powered applications make it an essential part of the electrical design of mobile robots.

2.2.3 Power Consumption Analysis

Power consumption is a fundamental consideration in the electrical design of mobile robotic systems because it directly affects operating time, system stability, component selection, and overall efficiency. A mobile robot typically consists of multiple electrical and electronic subsystems, such as embedded controllers, onboard computers, sensors, motor drivers, communication modules, and audio or display interfaces. Each subsystem consumes electrical power according to its voltage and current requirements. Therefore, understanding and estimating the total power demand of the robot is essential during the design stage.

In electrical systems, power is defined as the rate at which electrical energy is

consumed by a load. The basic relationship between voltage, current, and power is expressed by:

$$P = V \times I \tag{2.12}$$

The total power consumption of a robotic platform can be expressed as the sum of the power consumed by all electrical loads:

$$P_{total} = \sum_{i=1}^{n} (P_i) \tag{2.13}$$

1. Static Power Consumption: Static power consumption refers to the relatively constant power drawn by components that operate continuously, such as microcontrollers, onboard computers, LiDAR sensors, and communication modules.
2. Dynamic Power Consumption: Dynamic power consumption refers to the variable power demand caused by components whose load changes during operation, particularly motors and actuators. For example, a robot may consume significantly more power while accelerating, turning, or carrying payloads than when it is idle or moving at constant speed.

The operating time of a battery-powered robot can be roughly estimated using the battery energy capacity and the average system power consumption. Battery energy is commonly expressed in watt-hours (Wh), which can be calculated as:

$$E = C \times V \tag{2.14}$$

The estimated runtime of the robot can then be calculated as:

$$T = \frac{E}{P_{total}} \tag{2.15}$$

In summary, power consumption is a key parameter in the design of mobile robotic systems. It influences battery selection, operating time, voltage regulation, thermal performance, and system reliability. A thorough understanding of voltage, current, and power relationships enables designers to estimate energy demand accurately

and develop a stable electrical architecture suitable for continuous robotic operation.

2.2.4 Electrical Interconnection and Power Distribution

Electrical interconnection and power distribution are essential aspects of robotic electrical design because they determine how electrical energy is delivered from the power source to each subsystem in a safe, stable, and efficient manner. In a mobile robot, multiple components such as embedded controllers, onboard computers, sensors, motor drivers, audio modules, and communication devices operate simultaneously, often with different voltage and current requirements. Therefore, an appropriate electrical interconnection design is necessary to ensure that each subsystem receives the correct supply voltage and sufficient current for reliable operation.

Power distribution must also be designed to minimize undesirable effects such as voltage drop, electrical noise, and overheating. Voltage drop occurs when current flows through wires or connectors with finite resistance, reducing the effective voltage reaching the load. This can become significant in systems with long wires or high-current devices such as motors. To reduce voltage drop, designers typically use conductors with appropriate wire gauge, minimize unnecessary cable length, and ensure good-quality electrical connections. Stable voltage delivery is especially important for sensitive electronics such as microcontrollers, sensors, and onboard computers, which may malfunction if the supply voltage falls below their required operating range.

Electrical noise is another major concern in robotic systems, especially when low-power digital electronics and high-power actuators share the same power source. Motors, switching converters, and communication devices can introduce noise and transient disturbances into the power line. These disturbances may degrade sensor readings, interfere with communication signals, or affect processor stability. For this reason, electrical interconnection design often includes filtering capacitors, decoupling capacitors, proper grounding strategies, and physical separation between power and signal wiring. In some cases, sensitive subsystems are provided with isolated or separately regulated power lines to improve reliability. Thermal management is also a critical aspect of power distribution. High-current components such as motor drivers and voltage reg-

ulators can generate significant heat during operation. Proper heat dissipation through heatsinks, ventilation, or active cooling may be necessary to prevent overheating and ensure long-term reliability.

In addition, the power distribution architecture must consider the different operational characteristics of system components. High-current devices such as motors may produce sudden current surges during startup or direction changes, while computing units and sensors typically require stable, low-noise power supplies. Therefore, the electrical design should allocate power paths according to the characteristics of each load. Separating high-power and low-power branches can improve system stability and reduce interference between subsystems.

In summary, electrical interconnection and power distribution form the foundation of a reliable robotic electrical system. Proper distribution of power from the battery to all subsystems ensures that each component receives the required voltage and current while maintaining overall safety, stability, and efficiency.

2.3 SOFTWARE DESIGN

2.3.1 Mobile Robot

2.3.1.1 ROS 2 (Robot Operating System 2)

The Robot Operating System 2 is widely used in modern robotics because of its modular and distributed system design. In this framework, a robot system is divided into multiple nodes, where each node is responsible for a specific task and communicates with other nodes through message passing. This approach allows complex robotic systems to be organized into smaller and more manageable components. Many studies show that this structure improves system flexibility, scalability, and ease of maintenance. In addition, ROS2 uses Data Distribution Service (DDS) as its communication middleware, which supports reliable and real-time data exchange. This makes ROS2 suitable for applications that require continuous communication between sensors, processing units, and actuators [6].

2.3.1.2 Navigation Framework and Nav2 Stack

The Navigation2 is a commonly used framework in ROS2 for autonomous navigation. It provides a complete solution that includes path planning, obstacle avoidance, and motion control [7]. In general, the navigation system is divided into two main parts, which are global planning and local planning. The global planner generates a path from the robot's current position to a target location based on a known map, while the local planner adjusts the robot's movement in real time according to sensor data. Research has shown that the Dynamic Window Approach, implemented as the DWB controller, is effective for local navigation because it allows the robot to avoid obstacles while maintaining smooth motion. This makes Nav2 suitable for indoor autonomous robots.

2.3.1.3 Odometry and Sensor Fusion

Odometry is an important method used to estimate the position of a robot by calculating its movement over time. Encoder-based odometry uses wheel rotation data to estimate displacement, but it often accumulates errors due to wheel slip or uneven surfaces. To improve accuracy, many systems combine encoder data with information from an inertial measurement unit (IMU). The IMU provides data on acceleration and angular velocity, which can be used to estimate the robot's orientation. By combining multiple sensor sources, the system can produce a more reliable estimate of the robot's position. This process, known as sensor fusion, is widely used in robotics to improve localization performance.

2.3.1.4 LiDAR-Based Perception and Scan Matching

LiDAR sensors are commonly used in robotic systems to detect the surrounding environment because they provide accurate distance measurements and a wide scanning range. The RPLIDAR A1 is a popular sensor for indoor applications due to its affordability and 360-degree scanning capability. In addition to basic scanning, scan matching techniques are used to estimate the robot's movement by comparing consecutive LiDAR scans. The Canonical Scan Matcher is one example of this approach, where the system calculates the transformation between two scans by minimizing the

difference between matching points. This method improves motion estimation, especially in situations where encoder data alone is not reliable.

2.3.1.5 Localization using AMCL

The Adaptive Monte Carlo Localization is a widely used method for estimating a robot's position on a known map. This algorithm is based on a particle filter, where multiple possible positions are represented as particles. Each particle is updated based on sensor data and the robot's motion, and over time, the particles converge to the most likely position. Research shows that AMCL is effective in handling uncertainty and noise in sensor data. It is commonly used together with LiDAR sensors in indoor environments because it provides accurate and stable localization results.

2.3.1.6 Path Planning and Autonomous Navigation

Path planning is a key component of autonomous navigation, where the robot must find a safe path from its current location to a target position. In many systems, global planning algorithms such as A* or Dijkstra are used to compute an optimal path on a map. After that, local planning is used to adjust the robot's movement based on real-time sensor data. This combination allows the robot to avoid obstacles while still following the planned path. Studies show that using both global and local planning improves navigation performance and allows the robot to operate effectively in dynamic environments.

2.3.1.7 State Machine for Robot Behavior Control

Finite State Machines (FSMs) are commonly used to control robot behavior by defining different states and transitions between them. In this approach, the robot can switch between states such as idle, exploration, and goal-directed navigation depending on the situation. This method helps organize complex behaviors into a clear and structured system. Many studies show that FSM-based control improves system reliability and makes it easier to manage different robot operations, especially when integrating multiple subsystems.

2.3.1.8 Human-Robot Interaction via Web Integration

In modern robotic systems, integration with external services is becoming more important, especially for human-robot interaction. Web frameworks such as FastAPI are often used to create REST APIs that allow communication between robots and external servers. This approach enables the robot to receive commands, send status information, and connect with AI-based systems. It also allows for more flexible system design, where high-level processing can be handled outside the robot.

2.3.1.9 Communication Protocols in Robotics

Reliable communication between hardware and software components is essential in robotic systems. The Modbus RTU is widely used for this purpose because it is simple and reliable. It is commonly used in industrial applications to communicate with motor controllers and sensors. In robotic systems, Modbus RTU allows the robot to send control commands to motors and receive feedback data such as encoder values and sensor readings. This ensures stable and consistent operation of the robot hardware.

2.3.2 Database Management

2.3.2.1 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is a technique that enhances large language model output quality by incorporating an external knowledge retrieval step before response generation. Rather than relying solely on patterns in the model's training parameters, RAG queries a domain-specific knowledge base at inference time, retrieving contextually relevant information to augment the model's context.

RAG offers cost-effective extension of LLM capabilities to specialised domains without expensive model retraining. By grounding responses in retrieved, current information, RAG substantially reduces factual hallucinations and improves relevance and accuracy. In this implementation, RAG forms the backbone of the knowledge retrieval subsystem, enabling contextually appropriate responses to user queries about the op-

erating environment and domain-specific topics.

2.3.2.2 Vector Database

A vector database stores data as dense numerical vectors (embeddings) rather than conventional row-column structures, enabling similarity-based retrieval over large unstructured data collections including text, images, and audio. Vector databases support approximate nearest-neighbour search rather than exact-match queries, enabling retrieval of semantically related records even with differing query wording. This functionality is essential for RAG systems where semantic relevance rather than lexical overlap determines retrieval quality.

2.3.2.3 Embeddings

Embeddings are dense, fixed-dimensional numerical representations of objects — including text, images, and audio — produced by machine learning models trained to capture semantic and contextual relationships. Objects that are semantically similar are mapped to nearby points in the embedding space, enabling distance-based similarity measures to serve as a proxy for meaning. In this system, text embeddings represent both stored knowledge base documents and incoming user queries, enabling the vector database to identify and retrieve the most relevant passages for augmenting the LLM's response.

2.3.2.4 Milvus

Milvus is an open-source vector database platform specifically engineered for large-scale similarity search on massive embedding collections. Designed with production deployment in mind, Milvus provides highly optimised indexing structures and query algorithms that enable fast, accurate retrieval of semantically similar vectors from datasets containing millions or billions of embeddings. Unlike traditional relational databases, Milvus is purpose-built for vector search operations, supporting multiple similarity metrics including Euclidean distance, cosine similarity, and inner product.

Key features of Milvus include exceptional scalability through distributed ar-

chitectures that allow horizontal expansion, high-performance approximate nearest-neighbour search capabilities that retrieve relevant results with minimal latency even on very large datasets, and flexible indexing strategies optimised for different trade-offs between search accuracy and computational efficiency [8]. In this implementation, Milvus serves as the underlying vector database infrastructure for the RAG pipeline, enabling rapid semantic retrieval of domain-relevant information from the knowledge base to augment LLM responses.

2.3.2.5 SQL (Structured Query Language)

SQL is a standardised declarative programming language for defining, manipulating, and retrieving data in relational database management systems (RDBMS). Data is organised in tables with rows representing records and columns representing typed attributes. SQL provides comprehensive operations for schema creation, record insertion and updates, referential integrity enforcement, and complex multi-table queries through joins and aggregations. Query optimisation and index management facilities ensure retrieval performance at scale. In this system, SQL maintains structured records of user interactions, profiles, and system logs, complementing the vector database layer for semantic retrieval.

2.3.3 Artificial Intelligence

2.3.3.1 Face Recognition

Facial recognition is a biometric identification technology that authenticates or identifies individuals by analysing distinctive geometric and photometric features of the human face [9]. It detects facial regions, generates compact feature representations, and compares them against stored facial templates [10]. The system automatically identifies returning users and personalises interactions based on stored preferences and interaction history. This capability enhances human-robot interaction naturalness and continuity by eliminating explicit identification steps. For the robotic assistant, facial recognition enables context-aware, personalized engagement with individual users.

Modern face recognition systems are implemented as a multi-stage pipeline

comprising face detection, multi-object tracking, frontal face filtering, and embedding-based identity matching [11]. A face detector first identifies the locations of faces within each video frame, producing bounding boxes together with facial landmark points. Multi-object tracking then associates detections across consecutive frames, assigning a persistent track identifier to each individual to maintain identity continuity through brief occlusions and periods of low detection confidence. A head pose estimator computes the yaw, pitch, and roll angles of each tracked face from the landmark coordinates; only faces satisfying a frontal orientation constraint are eligible for recognition, preventing degraded embedding quality caused by profile or extreme off-angle views. Eligible face crops are passed to a recognition embedding model that produces a compact, fixed-dimensional numerical representation of facial appearance. Identity is then determined by comparing the query embedding against a gallery of stored enrolment embeddings using a similarity metric such as cosine similarity, returning a known identity when the similarity exceeds a configurable threshold, or assigning an unknown label otherwise. This multi-stage design balances computational efficiency with recognition accuracy in real-time robotic deployment scenarios.

2.3.3.2 Natural Language Processing (NLP)

Natural Language Processing (NLP) is a multidisciplinary field enabling computational systems to understand, interpret, and generate human language in written and spoken forms [12]. It combines rule-based linguistics, statistical models, and deep learning architectures for tasks including parsing, sentiment analysis, intent detection, and machine translation [13]. NLP systems capture not only surface language form but also deeper semantic content, including speaker intent, implied meaning, and emotional tone. This nuanced understanding enables the robotic assistant to engage in natural, contextually appropriate dialogue, especially in open-domain settings with ambiguous or colloquial queries.

2.3.3.3 Large Language Model (LLM)

A Large Language Model (LLM) is a class of artificial intelligence system trained on massive corpora of text data — often comprising hundreds of billions of tokens — to acquire broad and deep competencies in understanding and generating human language [14]. LLMs are built on the transformer neural network architecture, which leverages self-attention mechanisms to capture long-range dependencies between tokens and model the statistical structure of language at multiple levels of abstraction.

Through large-scale pre-training followed by task-specific fine-tuning or instruction alignment, LLMs develop the capacity to perform a diverse array of language tasks — including question answering, summarization, translation, and code generation — without requiring explicit task-specific training data for each application [15]. In this system, the LLM serves as the central reasoning and response generation component of the conversational AI subsystem, processing retrieved contextual information from the RAG pipeline to produce coherent, factually grounded responses to user queries [16].

2.3.3.4 Text-to-Speech (TTS)

Text-to-Speech (TTS) is a speech synthesis technology that converts written text into natural-sounding spoken audio, enabling AI systems to communicate verbally with users without requiring pre-recorded audio assets. Modern TTS systems are built on deep learning architectures — including neural vocoders and sequence-to-sequence models — that generate speech with dynamic, context-sensitive variations in prosody, pitch, speaking rate, and pronunciation. These characteristics produce output that closely approximates natural human speech patterns, significantly improving comprehensibility and user engagement compared with earlier rule-based synthesis methods [17], [18].

TTS technology has found widespread application in accessibility tools for visually impaired users, virtual assistants, and interactive voice response systems [19]. For the robotic assistant, TTS serves as the primary output modality for verbal communi-

cation, enabling the robot to deliver responses audibly in environments where visual screen interaction is impractical or undesirable [20].

2.3.3.5 Automatic Speech Recognition (ASR)

Automatic Speech Recognition (ASR) is a technology that processes raw acoustic speech signals and converts them into machine-readable text, forming the primary input pathway for voice-driven human-machine interaction [21]. Contemporary ASR systems employ end-to-end deep learning architectures — including recurrent neural networks and transformer-based models — trained on large-scale speech corpora to achieve high transcription accuracy across a range of speakers, accents, and acoustic conditions [22].

Advanced ASR implementations integrate Natural Language Processing modules downstream of the transcription stage, enabling the system to infer the user's intent and semantic meaning from the recognised text in addition to producing a verbatim transcript. ASR accuracy is sensitive to several environmental factors, including background noise levels, microphone quality, and speaker proximity; consequently, robust noise handling and signal preprocessing are important design considerations for deployment in real-world robotic environments. Despite these challenges, modern ASR systems are capable of achieving near real-time transcription latency, enabling fluid conversational interactions between the user and the robotic assistant [23].

2.3.3.6 Voice Activity Detection (VAD)

Voice Activity Detection (VAD) is a technology that automatically identifies the presence or absence of human speech within an audio signal. It serves as a critical preprocessing stage in speech processing pipelines, enabling downstream modules such as automatic speech recognition to process only speech-containing segments while discarding silence, background noise, and non-speech acoustic events. By filtering out non-speech audio before ASR inference, VAD reduces unnecessary computational load and prevents the hallucination of spurious transcribed text from ambient sounds or environmental noise.

Traditional VAD approaches relied on energy thresholding or zero-crossing rate estimation, which are sensitive to noise and require manual parameter tuning for each deployment environment. Contemporary VAD systems employ deep neural networks trained on large and diverse speech corpora to output a probability score for each audio frame, indicating the likelihood that the frame contains speech. Frames with probability values exceeding a configurable threshold are classified as speech and forwarded to the speech recognition backend, while frames below the threshold are discarded.

In robotic systems deployed in indoor environments, VAD is especially important for suppressing hallucination loops in neural ASR models, which can generate repeated words or phrases when continuously presented with background noise or silence. A two-tier VAD pipeline — combining a pre-emphasis filter to amplify high-frequency speech components with a neural probability-based VAD model — provides robust and reliable speech detection in acoustically challenging conditions without the need for keyword or wake-word detection mechanisms.

2.3.3.7 Multi-Object Tracking

Multi-object tracking (MOT) is the computer vision problem of jointly detecting and maintaining the identities of multiple objects across a sequence of video frames. In robotic perception systems, tracking is used to associate detections across time, enabling the system to follow individual persons or objects even when they temporarily leave the field of view or are occluded by other scene elements.

Contemporary tracking algorithms typically follow the tracking-by-detection paradigm, in which a detector is applied independently to each frame and a subsequent data association step links detections to existing tracks using geometric or appearance-based cues. High-performance tracking methods such as ByteTracker extend conventional association by incorporating low-confidence detection boxes in a secondary matching pass, recovering objects that are partially occluded or temporarily invisible and maintaining more stable trajectories across challenging sequences. Each tracked individual is assigned a persistent track identifier that remains consistent across frames, enabling downstream processes — such as face recognition and interaction management — to

apply per-identity cooldowns, accumulate recognition evidence over multiple frames, and gate interaction triggers to prevent false activations from brief or transient detections.

CHAPTER 3

METHODOLOGY

3.1 SYSTEM OVERVIEW

The Satu robotic assistant is implemented as a distributed embedded architecture consisting of four computational nodes: a central AI Brain server, an NVIDIA Jetson Orin Nano vision module, and two Raspberry Pi 5 edge devices. This distributed design balances computational workload while minimising latency across perception, reasoning, and actuation subsystems.

The AI Brain server acts as the core intelligence layer, operating a FastAPI-based system that manages the entire interaction pipeline. It integrates large language model (LLM) inference via Ollama, a Retrieval-Augmented Generation (RAG) pipeline, speech-to-text (STT) and text-to-speech (TTS), and database services. A hybrid storage system is employed, combining Milvus for vector-based semantic retrieval and MySQL for structured relational data. Additionally, SQLite is used for lightweight conversation logging and system monitoring.

The Jetson Orin Nano module is responsible for real-time face detection and recognition, leveraging GPU acceleration through TensorRT-optimised ONNX models to perform biometric identification of registered users with low latency.

The first Raspberry Pi 5 functions as the audio interface and coordination node, handling microphone input, performing voice activity detection and on-device speech transcription, and playing back synthesised speech responses received from the server. It also hosts the robot's expressive face display, rendered on an HDMI screen using an Electron.js application. The second Raspberry Pi 5 operates as the mobility controller, executing ROS2-based navigation, including path planning, wheel control, and obstacle avoidance.

3.1.1 Distributed Node Architecture

The four computational nodes are connected over a local area network and communicate via a combination of persistent WebSocket channels and stateless HTTP

Table 3.1 System Components and Specifications

Component	Specification
AI Brain Server	Ubuntu 22.04 LTS, NVIDIA GPU
Raspberry Pi 5 #1 (Audio + Display)	Raspberry Pi OS (64-bit)
Raspberry Pi 5 #2 (Navigation)	Raspberry Pi OS (64-bit)
Jetson Orin Nano	NVIDIA Jetson Orin Nano (8 GB)
LiDAR Sensor	RPLiDAR A1 (2D, 360°)
Drive Controller	iREDCr Differential Drive
Navigation Middleware	ROS2 Jazzy Jalisco + Nav2
LLM Inference	Ollama (Typhoon2-70B, Q5_K_M GGUF)
Vector Database	Milvus v2.6
Relational Database	MySQL + SQLite
Audio Processing	Typhoon2-Audio 8B (STT + TTS sidecar)
Face Recognition	SCRFD + ArcFace (ONNX / TensorRT)

requests. The network topology is intentionally hub-and-spoke: the AI Brain server acts as the central coordinator. All edge nodes communicate solely with this server rather than directly with each other, except for a dedicated Jetson-to–Raspberry Pi 5 WebSocket connection that enables low-latency transmission of face detection events.

Figure 3.1 illustrates the distributed node topology and the primary communication links between all four computational nodes.

Table 3.2 Network Endpoints and Communication Protocols

Host	IP Address	Port	Protocol	Service
AI Brain Server	10.100.16.22	8000	HTTP	Main FastAPI Service
AI Brain Server	10.100.16.22	8001	HTTP	Typhoon2-Audio Sidecar
AI Brain Server	10.100.16.22	19530	gRPC	Milvus Vector Database
Jetson Orin Nano	192.168.1.10	9999	TCP	Remote Shutdown Listener
Pi 5 #1 (Audio)	10.26.9.196	8765	WebSocket	Jetson Detection Receiver
Pi 5 #1 (Audio)	10.26.9.196	8766	HTTP	RaspiReceive FastAPI
Pi 5 #1 (Display)	10.26.9.196	7000	HTTP	Face Emotion API
Pi 5 #1 (Internal)	127.0.0.1	8768	WebSocket	Face UI IPC Bridge
Pi 5 #2 (Navigation)	10.26.9.196	8767	HTTP	ROS2 Navigation Bridge

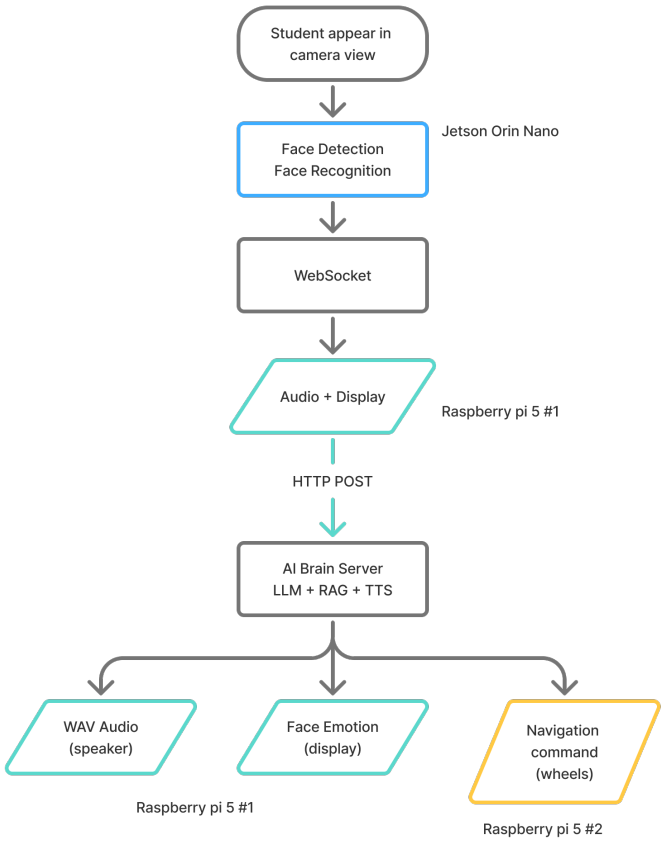


Figure 3.1 System Overview: Distributed Node Architecture and Communication Pathways

3.1.2 End-to-End Interaction Pipeline

The complete pipeline for a single human-robot interaction follows a linear sequence of processing stages:

1. The Jetson Orin Nano continuously captures camera frames and applies face detection and recognition. Upon identifying a student or an unidentified visitor, it transmits a detection event as a JSON message over the persistent WebSocket to Pi 5 #1.
2. Pi 5 #1 receives the detection event and simultaneously records microphone audio. A Voice Activity Detection model gates speech capture, and a speech recognition model transcribes the captured segment.
3. The coordination logic on Pi 5 #1 combines the detection event with the transcription result and dispatches a single JSON payload to the AI Brain server via an HTTP POST request.
4. The AI Brain server executes the full inference pipeline: grammar correction, query routing, RAG retrieval, LLM response generation, and TTS synthesis.
5. The server dispatches responses asynchronously: synthesised audio (WAV) to Pi 5 #1 for playback, a microphone activation flag (/set_active) to the audio controller, and when required, a navigation command to Pi 5 #2. Face emotion transitions are managed locally by Pi 5 #1 based on its own audio playback lifecycle.

3.2 MECHANICAL DESIGN AND CONSTRUCTION

3.2.1 Overall Body Structure

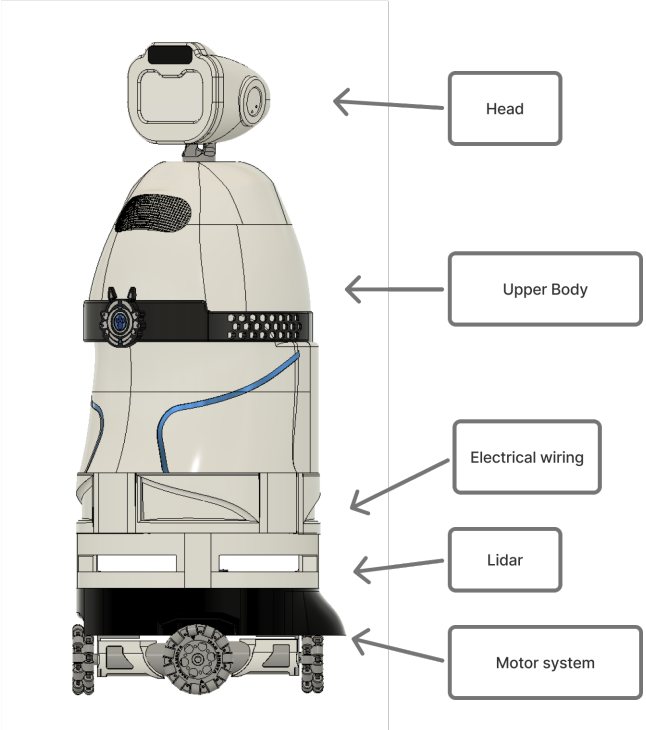


Figure 3.2 Overall Body Design

The robot was separated into the base part and the upper body part to simplify structural design, fabrication, and maintenance. The base provides mobility and support for the lower subsystems, while the upper body provides space for the main enclosure and upper components. This two-part arrangement also improves accessibility during assembly and servicing.

3.2.1.1 Base Structure

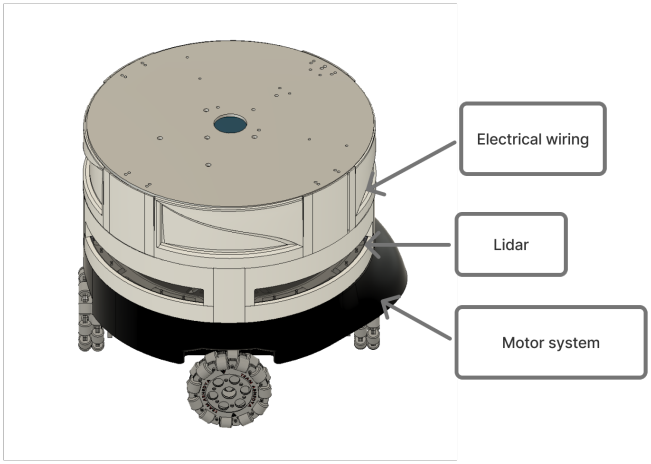


Figure 3.3 Base Part Design

The robot base was designed as the main mobile platform and was divided into three structural layers. The first layer contains the wheels, motors, motor driver, and microcontroller. The second layer houses the LiDAR sensor at the center of the robot. The third layer is used for the electrical section and also acts as the top support plate for the upper body. This layered arrangement helps organize the internal components according to their functions while maintaining a compact structure. The base structure was designed to be as low-profile as possible to improve stability and reduce the risk of tipping during movement.

3.2.1.2 Upper Body Structure

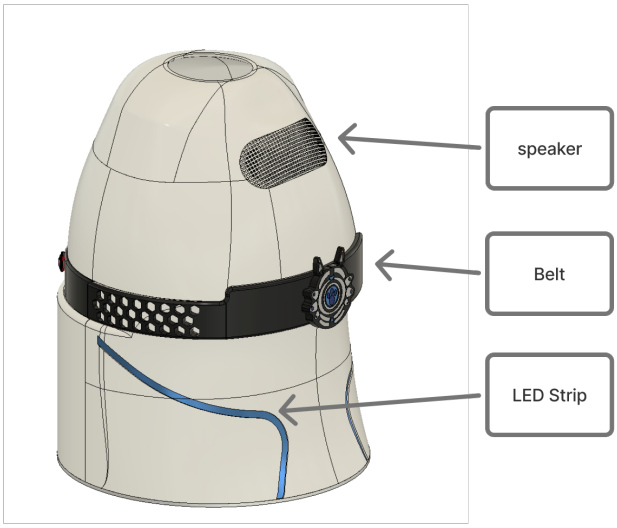


Figure 3.4 Upper Body Design

The upper body was designed as the main exterior enclosure of the robot. It accommodates the battery stack, head support, speaker support, and user interface components. The upper body shell is mounted directly onto the base top plate. A central aluminum profile is used to support the head and nearby speaker structure, while the rear section contains the stacked battery arrangement. The overall shape of the upper body is streamlined to provide a friendly and approachable appearance while also allowing sufficient internal space for all necessary components.

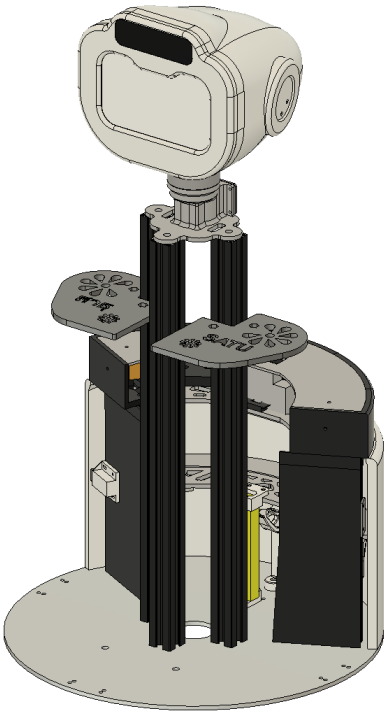


Figure 3.5 Upper Body Interior Structure

3.2.2 Exterior Casing Design

The exterior casing was designed to protect internal components while creating a suitable appearance for a service robot. A rounded body form was selected to give the robot a softer and friendlier look. Smooth surfaces and curved transitions were used to reduce the mechanical appearance of the structure. The front section of the upper body was designed to accommodate the display screen for the robot's face, while the rear section was shaped to house the battery stack. The overall design aimed to balance aesthetics with functionality, ensuring that all components are securely enclosed while maintaining an inviting presence.

The lower torso includes a side curve that was mainly introduced for aesthetic purposes. This curve helps reduce the visual bulk of the body and creates a more dynamic shape than a plain cylindrical form. An LED strip was placed along this curve to emphasize the body contour and enhance the visual appearance of the robot.

Some casing parts were designed to be removable for maintenance purposes. These include the battery and small cutout case in the upper body, the electrical

wall, and the two rear lower covers. This arrangement allows easier access to internal components without fully dismantling the robot body.

3.2.2.1 Head Casing Design

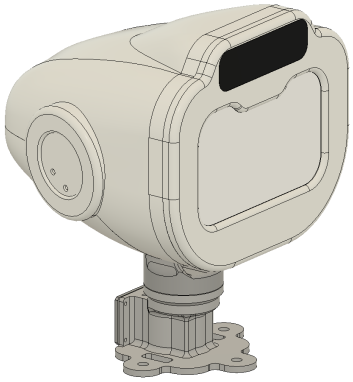


Figure 3.6 Robot Head Design

The head casing was designed with a rounded form consistent with the overall body style. This design helps create a unified appearance between the head and torso. The head is mounted on a printed base connected to the central support structure of the upper body. The head shell is designed to be easily removable for maintenance and component access. The front of the head includes a cutout for the camera module, while the rear section provides space for the speaker and associated wiring. The head design prioritizes both aesthetics and functionality, ensuring that all components are securely housed while maintaining a friendly and approachable look.

3.2.3 Materials and Fabrication

3.2.3.1 Aluminum

Aluminum was selected as the main structural material due to its light weight, practical rigidity, and ease of fabrication. The base uses 5 mm aluminum plates and 6063 aluminum profiles as the main supporting structure. The structural materials were selected based on their common use in lightweight robotic structures, practical rigidity, and suitability for the expected load conditions

3.2.3.2 Acrylic

Acrylic was used in parts that required electrical insulation and flat support surfaces. In the base, a 5 mm acrylic sheet was added in the electrical section to reduce the risk of short circuits. In the upper body, acrylic was used as the base of the upper battery tray because it is economical and can be fabricated quickly by laser cutting. The acrylic sheet provides sufficient strength to support the battery weight while also electrically isolating it from the aluminum structure.

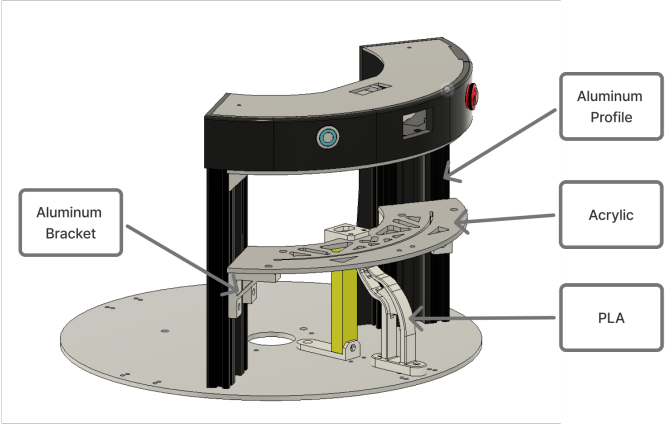


Figure 3.7 Material in Battery Tray Design

3.2.3.3 3D-Printed PLA

PLA was selected for most casing parts because it is lightweight and easier to print using the available FDM settings compared with ABS or PETG. It is also suitable for producing curved surfaces, which are important in the exterior design of the robot. Due to the large size of the upper body, the shell was divided into smaller printable parts. The head shell was printed as a single piece to maintain the smooth curvature and avoid visible seams. The PLA parts were designed with sufficient thickness and infill to ensure durability while keeping the overall weight manageable.

3.2.3.4 Fabrication Process

- Structural Part:The base frame was constructed using 5 mm aluminum plates connected by 20 × 20 mm 6063 aluminum profiles. Eight aluminum profile pillars were used to support the three-layer base structure. This frame provides the main load-bearing support for the wheels, motors, control components, LiDAR, and electrical section.
- Exterior Parts :The exterior casing parts were fabricated from PLA using the FDM process. This method was selected because it is suitable for producing the curved outer surfaces of the robot. Large upper body parts were divided into smaller sections to fit the printing size limitation.
- Bonding and Surface Finishing: The printed upper body parts were joined using epoxy adhesive together with internal tabs and alignment features. After joining, the surface was finished by sanding, putty application, sanding again, and spray painting. This process helped improve surface smoothness and reduce the visibility of seams between printed parts.

3.3 ELECTRICAL SYSTEM DESIGN

3.3.1 Power System Design

The overall power supply architecture of the robot system is illustrated in Figure 3.8. The design integrates multiple voltage regulators and distributes power to key subsystems including the Raspberry Pi, sensors, and actuators.

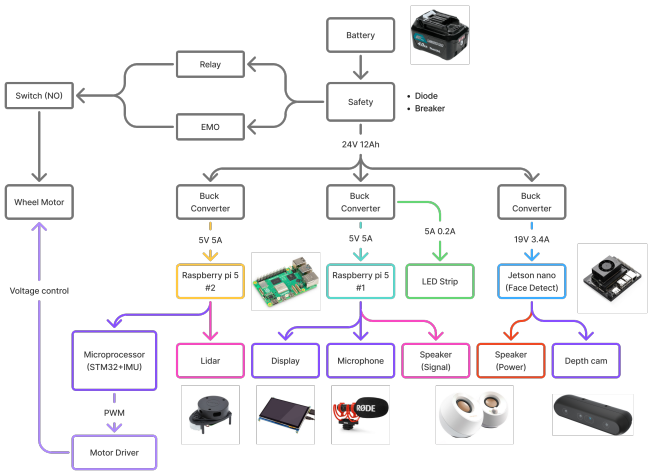


Figure 3.8 Overview of Power Supply Architecture

The robot is powered by 6 of 12 V, 4 Ah Lithium-ion batteries arranged as three parallel branches, where each branch consists of two batteries connected in series to produce 24 V. As shown in Figure 3.9, a diode is placed at the output of each battery branch before the branches are combined, in order to prevent reverse current flow between parallel branches. After the combined output, a breaker is installed as the main protection device to disconnect the circuit under abnormal electrical conditions. This battery and protection arrangement forms the primary power source stage before electrical energy is distributed to the motor system and the regulated electronic subsystems.

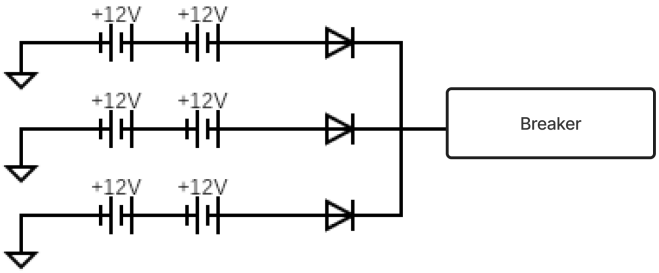


Figure 3.9 Battery circuit Schematics

The main battery output is divided into two functional branches. The first branch supplies the wheel motor system through a relay, an emergency stop (EMO) circuit, and a main motor switch. This arrangement allows the drive system to be controlled independently from the low-power electronic subsystems and improves operational safety by enabling rapid interruption of motor power when required. The second branch supplies the computing and sensor subsystems through several buck converters that generate the required regulated voltages.

Since the robot contains multiple subsystems with different voltage requirements, separate converters were used to generate suitable supply voltages for each major computing unit. In the current design, the robot uses two 5 V rails for the two Raspberry Pi 5 units and one 19 V rail for the Jetson Nano, while the wheel motors are supplied directly from the 24 V rail to maintain stable power delivery to sensitive devices.

In this design, the buck converters have an efficiency of 95

$$\eta = \left(\frac{P_{out}}{P_{in}}\right) x100\%$$

Using the estimated continuous loads from the regulated branches:

the required input power from the battery can be estimated as follows.

Table 3.3 Ideal Power of Raspberry Pi 5 and Jetson Orin Nano

Component	Estimated Power per Unit (W)
Raspberry Pi 5	25
Jetson Orin Nano	64.6

$$P_{in,Pi} = \frac{P_{out,Pi}}{\eta} = \frac{25}{0.95} \approx 26.32W$$

$$P_{in,Jetson} = \frac{P_{out,Jetson}}{\eta} = \frac{64.6}{0.95} \approx 68W$$

Thus, the total battery power required by these three buck-converted computing branches is:

$$P_{in,total} = (P_{in,Pi} \times 2) + P_{in,Jetson} = (26.32 \times 2) + 68 \approx 120.64W$$

This shows that although the total regulated output power is:

$$P_{out,total} = (25 \times 2) + 64.6 = 114.6W$$

the actual power drawn from the battery is higher due to the conversion loss:

$$P_{loss} = P_{in,total} - P_{out,total} = 120.64 - 114.6 = 6.04W$$

Therefore, the buck converter efficiency should be considered in the power distribution design because it affects the actual battery power demand, thermal performance, and effective operating time of the robot.

Overall, the power source, distribution, and interconnection design were developed to ensure that each subsystem receives the required operating voltage while maintaining safe, stable, and well-organized power delivery throughout the robot.

3.3.1.1 Power Consumption Estimation

Power consumption estimation was carried out to evaluate the expected electrical demand of the robot and to support the design of the battery system and power distribution architecture. In a battery-powered mobile robot, this analysis is important because it directly affects operating time, regulator selection, and overall system stability. In this project, the electrical loads were classified into continuous loads and intermittent loads according to their operating characteristics.

Continuous loads refer to components that remain powered during most of the robot operation. These include the onboard computing units, display, sensing devices, and LED strip. Intermittent loads refer to components that are activated only during certain functions or whose power demand varies depending on the operating condition. In this robot, the microphone, speaker, and wheel motors were treated as intermittent loads.

The electrical power of each component was estimated using:

$$P = V \times I$$

Table 3.4 Electrical Load and Power of Robot Components

Component	Voltage (V)	Max Current (A)	Power/Unit (W)	Qty	Total Power (W)	Load Type
Raspberry Pi 5	5	5	25	2	50	Continuous
Jetson Orin Nano	19	3.4	64.6	1	64.6	Continuous
LiDAR Sensor	5	0.1	1	1	1	Continuous
Microphone RØDE VideoMic GO II	5	0.1	0.5	1	0.5	Intermittent
7 inch HDMI Touch Screen Display	5	3	15	1	15	Continuous
Creative Pebble Speaker	5	2	10	1	10	Intermittent
Intel RealSense Depth Camera	5	0.8	4	1	4	Continuous
DC Motor	24	1.7	40	4	160	Intermittent

From table 3.3, the total estimated continuous and intermittent power consumption is:

$$P_{continuous} = 50 + 64.6 + 1.0 + 0.5 + 15.04.0 = 135.1W$$

Since the regulated branches use buck converters with an efficiency of 95

$$P_{continuous,in} = \frac{P_{continuous,out}}{\eta} = \frac{135.1}{0.95} \approx 142.21W$$

$$P_{intermittent} = 0.5 + 10.0 + 160 = 170.5W$$

The robot uses a battery pack configured from 6 Litium-ion batteries, giving a total supply of 24 V, 12 Ah. The total battery energy can therefore be estimated as:

$$E = V \times C = 24 \times 12 = 288W$$

The ideal operation time is estimated from continuous load component.

$$T_{continuous} = \frac{E}{P_{continuous}} = \frac{288}{135.1} \approx 2.13hours$$

However, when the 95

$$T_{continuous,practical} = \frac{E}{P_{continuous,in}} = \frac{288}{142.21} \approx 2.02hours$$

The worst-case operating time refers to the condition in which all electrical components are assumed to be active simultaneously, resulting in the maximum estimated total power consumption of the system.

$$T_{total} = \frac{E}{P_{continuous} + P_{intermittent}} = \frac{288}{135.1 + 170.5} \approx 0.94hours$$

Overall, the power consumption analysis indicates that the major continuous electrical demand of the robot originates from the onboard computing units, particularly the Jetson Nano and the two Raspberry Pi 5 boards. In contrast, the wheel motors constitute the largest intermittent load during robot motion. The worst-case operating time is defined as the condition in which all electrical components are assumed to

be active simultaneously, resulting in the maximum total power demand of the system. Therefore, this analysis provides a useful basis for evaluating the suitability of the battery configuration and for further improving the electrical design of the robot.

3.4 ROS SYSTEM ARCHITECTURE AND DEVELOPMENT

3.4.1 ROS Architecture Overview

The ROS2 system architecture in this project is designed to organize the robot software into multiple functional layers that work together to achieve autonomous navigation and human interaction. The overall system is structured in a layered pipeline, starting from the external AI server and progressing through the HTTP bridge, navigation manager, Nav2 stack, kinematics layer, and finally the motor controller.

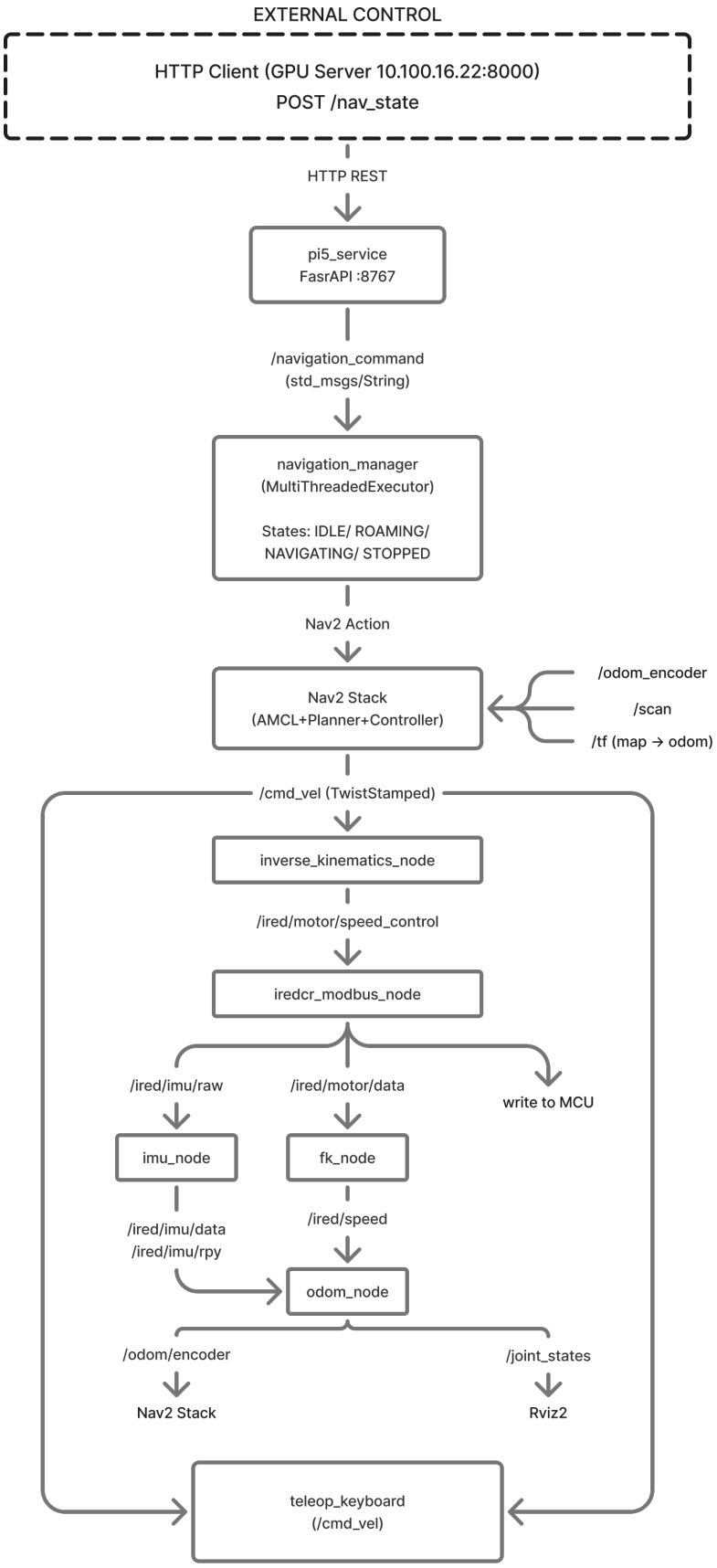


Figure 3.10 ROS Data Flow Diagram

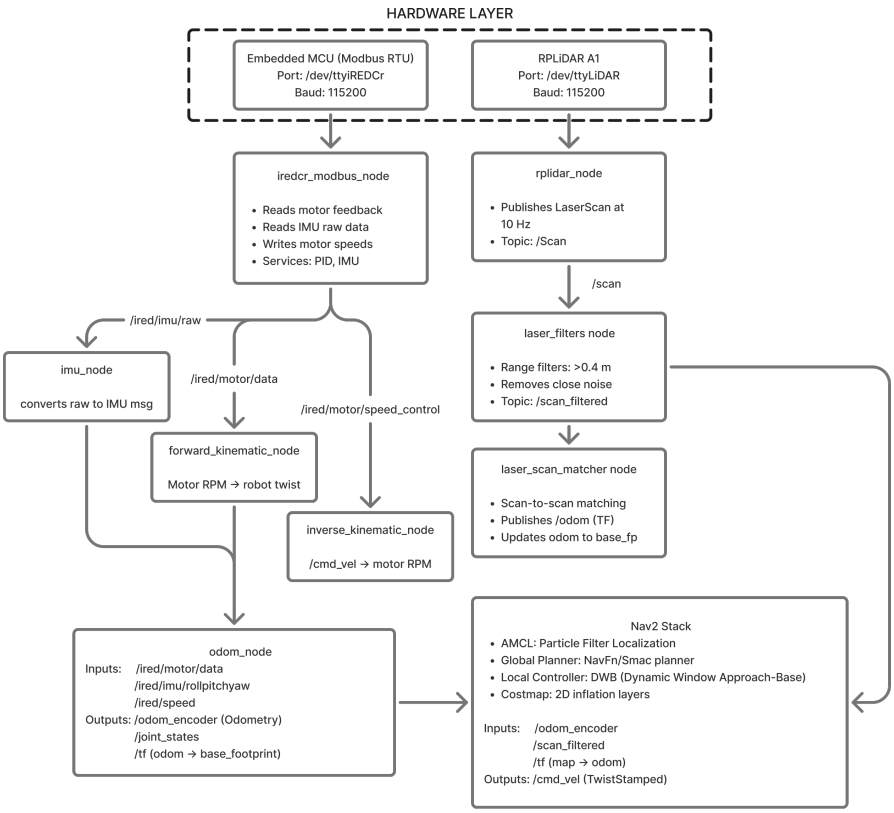


Figure 3.11 ROS Architecture Diagram

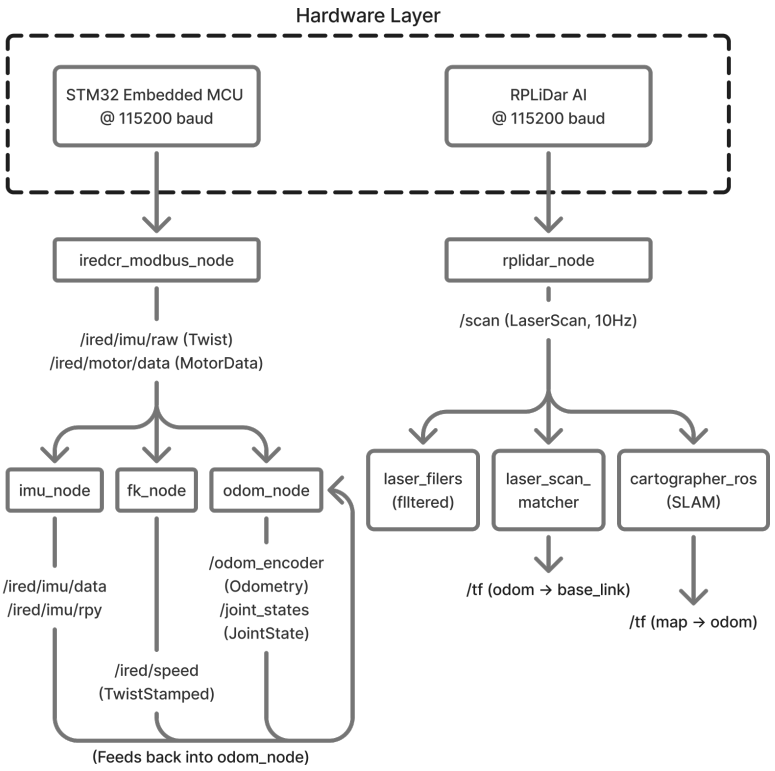


Figure 3.12 ROS Node Diagram

In this architecture, high-level commands from the AI server are gradually con-

verted into low-level motor actions. For example, navigation commands are first received via HTTP, then translated into ROS2 messages. These messages are processed by the navigation state machine, passed to the Nav2 stack for path planning, and then converted into wheel velocities using inverse kinematics. Finally, the computed motor speeds are sent to the motor controller through the Modbus RTU protocol.

At the same time, sensor data such as encoder and LiDAR readings flow in the opposite direction to support decision-making. This bidirectional communication allows the robot to adjust its behavior based on both external commands and real-time environmental feedback.

3.4.2 Hardware Communication and Motor Control

The hardware communication layer is handled by the iredcr_modbus_node in the ired_bringup package. This node is responsible for communication between the ROS2 system and the embedded motor controller using the Modbus RTU protocol over a serial interface.

The communication is configured at 115200 baud via the /dev/ttyiREDCr port. The node runs on a 10 ms loop, performing two main operations in each cycle:

1. Writing motor speed commands to the controller
2. Reading feedback data from the controller

The feedback includes four wheel encoder values and six IMU measurements (linear acceleration and angular velocity). These values are then published as ROS2 topics for further processing.

To ensure safety, a timeout mechanism is implemented. If no velocity command is received within one second, all motor speeds are set to zero automatically. This prevents unintended movement in case of communication failure. Additionally, an encoder validation check is applied to avoid incorrect zero readings by keeping the previous valid value when necessary.

3.4.3 Omnidirectional Kinematics

The robot uses a four-wheel mecanum drive system, which allows movement in all directions without changing orientation.

Two ROS2 nodes are used to handle kinematics:

1. Forward kinematics node:This node calculates the robot's velocity based on encoder feedback. Wheel speeds are converted from RPM to meters per second, and then combined to compute linear velocities (x, y) and angular velocity (z). The result is published as a TwistStamped message.
2. Inverse kinematics node: This node performs the opposite process. It receives velocity commands from /cmd_vel and converts them into individual wheel speeds, which are then sent to the motor controller.

3.4.4 Sensor Processing and Odometry

3.4.4.1 IMU Processing and Odometry Estimation

The IMU data from the motor controller is processed by the imu_node. This node converts raw sensor values into a standard ROS2 IMU message format.

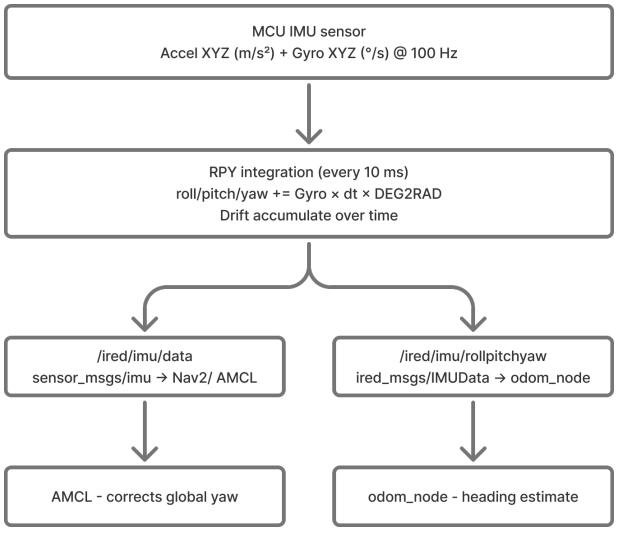


Figure 3.13 IMU Data Processing Diagram

The orientation (roll, pitch, yaw) is calculated by integrating angular velocity over time. The results are published both as Euler angles and quaternion format.

The odom_node uses both the velocity from forward kinematics and the ori-

entation from the IMU to estimate the robot's position. The position is updated every 10 ms by integrating velocity over time.

The computed odometry is published to /odom_encoder and used as the main motion estimation for the navigation system. This encoder-based odometry is later combined with LiDAR-based estimation to improve accuracy.

3.4.4.2 LiDAR Integration and Scan Filtering

The robot uses an RPLIDAR A1 sensor to detect its surroundings. The sensor is connected via serial communication and managed by the rplidar_ros package.

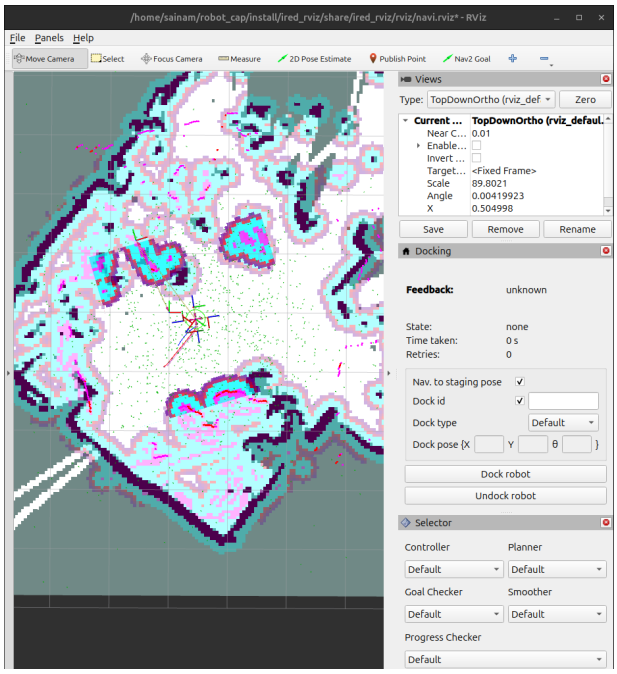


Figure 3.14 LiDAR Scan Visualization

The LiDAR provides 360-degree scan data, which is processed and published as a LaserScan message. To improve data quality, a filtering process is applied.

A minimum distance filter (0.40 m) is used to remove points that come from the robot's own structure. The filtered scan is then published to scan_filtered and used for both navigation and localization.

3.4.4.3 Scan Matching Odometry

The system uses the ros2_laser_scan_matcher package to estimate motion based on LiDAR data. This method uses the Canonical Scan Matcher (CSM) algorithm.

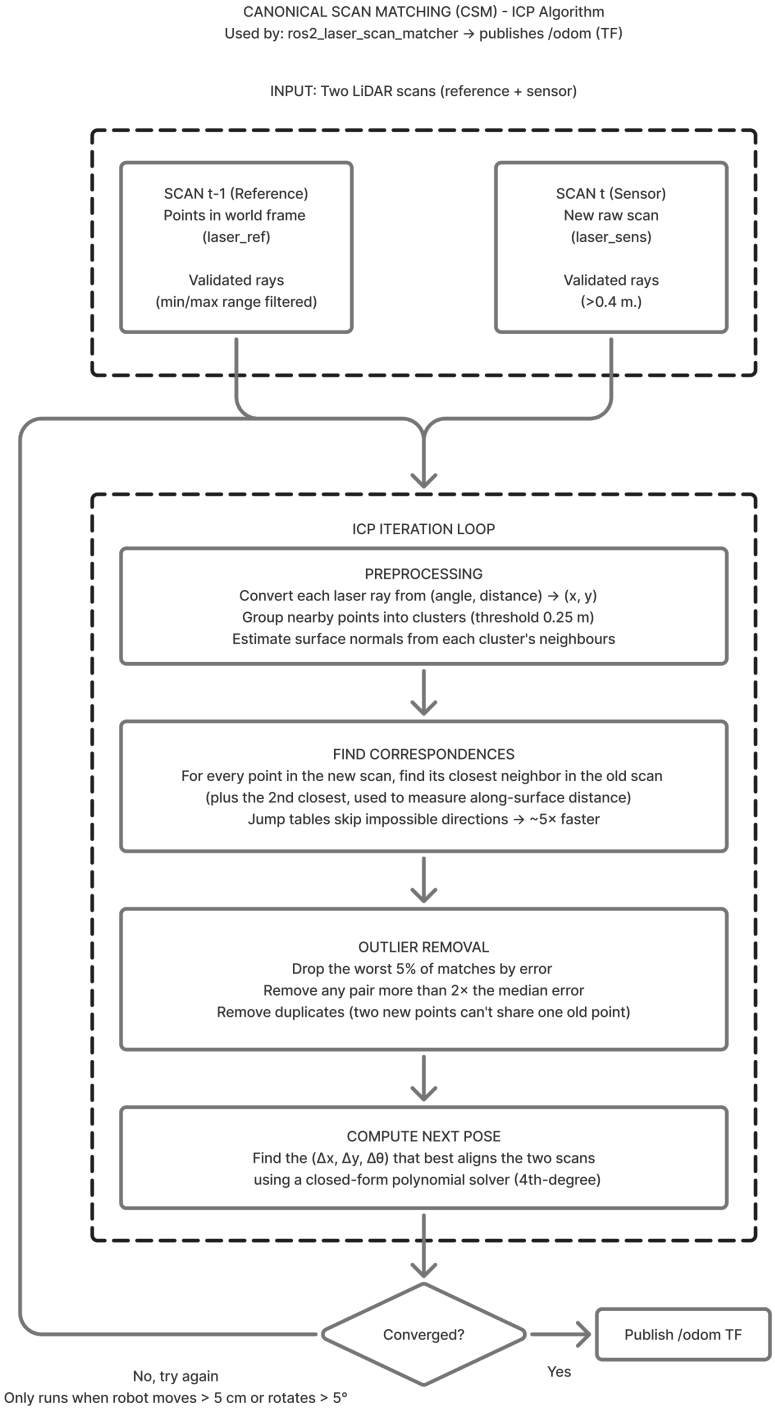


Figure 3.15 Scan Matching (ICP) Diagram

The algorithm compares consecutive scans and calculates the robot's movement by minimizing the distance between matching points. The result is an additional

odometry estimate published to /odom.

This method improves accuracy compared to encoder-only odometry, especially in environments with wheel slip.

3.4.5 Navigation Stack and Behaviour

3.4.5.1 Map-Based Localisation with AMCL

The robot uses Adaptive Monte Carlo Localization (AMCL) to estimate its position on a known map.

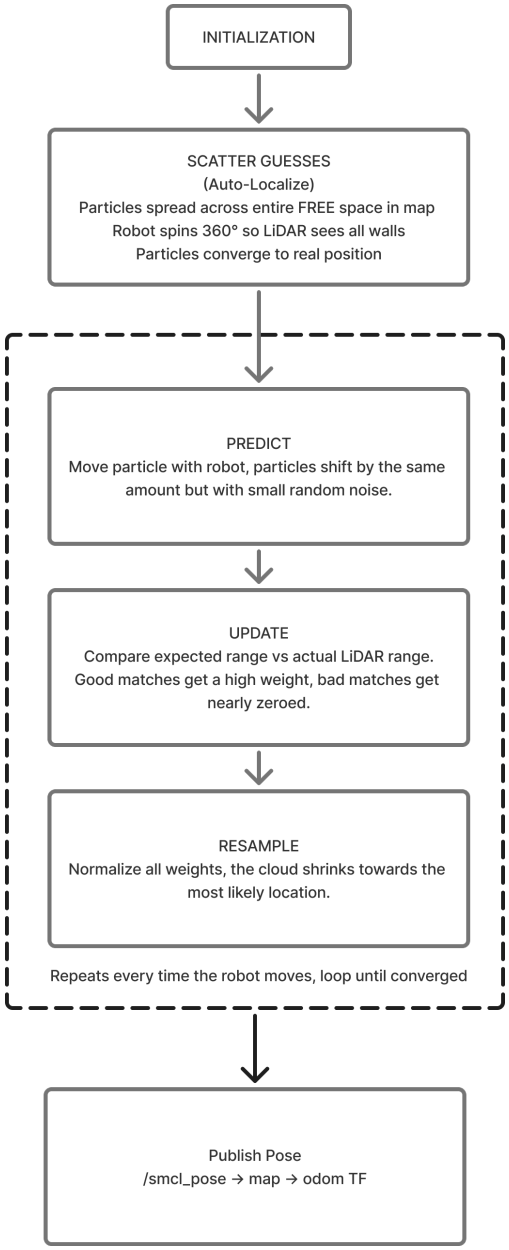


Figure 3.16 AMCL Particle Filter Diagram

AMCL uses a particle filter to represent possible robot positions. Each particle is updated based on LiDAR data, and over time, the particles converge to the correct position.

3.4.5.2 Auto-Localisation at Initial Position

To simplify operation, an automatic localization process is implemented.

At startup, the robot performs a 360-degree rotation while AMCL updates its estimate. This process runs for at least 15 seconds to ensure accuracy.

The robot executes one or more complete 360-degree rotation manoeuvres using the Nav2 spin behaviour, during which the LiDAR continuously sweeps all surrounding directions. As each scan is processed, AMCL matches the observed geometry against the stored map walls, causing the particle distribution to converge progressively toward the true robot pose.

3.4.5.3 Path Planning

Once localisation is established, the navigation system generates a collision-free path from the robot's current estimated pose to the target destination using the Nav2 stack.

The global planner computes an initial path over the occupancy grid map, while the local planner, implemented using the DWB controller, executes real-time trajectory optimisation within a rolling local costmap constructed from live LiDAR scans.

3.4.5.4 Stuck Detection and Recovery

During navigation, the system continuously monitors the robot's progress toward its goal by tracking the distance_remaining value provided in Nav2 action feedback at regular intervals.

If the robot does not move closer to the goal within 15 seconds, it is considered stuck. The system then:

1. Cancels the current goal
2. Performs re-localization

3. Retries the goal (up to 2 times)

This layered recovery strategy ensures that the robot can resume autonomous operation after transient failures without requiring external intervention.

3.4.5.5 Navigation State Machine

The navigation system is controlled by a finite state machine implemented in the NavigationManager node.

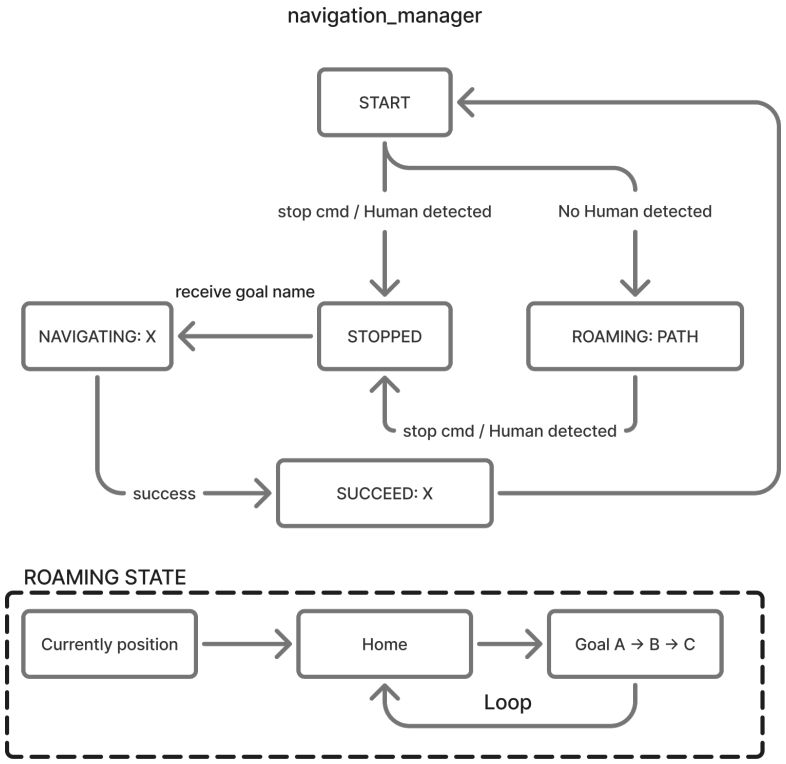


Figure 3.17 State Machine Diagram

3.4.5.6 Idle and Stop State

The idle or stop state represents the condition in which the robot remains stationary to support human interaction processes while awaiting further navigation commands. In this state, the robot temporarily suspends all movement-related operations to allow external interaction modules, such as the AI-based communication system, to engage with users without interruption.

3.4.5.7 Autonomous Roaming State

In the autonomous roaming state, the robot performs exploratory movement within the environment without a predefined destination, allowing continuous operation when no specific task is assigned. This behaviour is implemented through a predefined set of waypoints, where the robot navigates sequentially between locations in a looping manner to maintain coverage of the environment. At each waypoint, the robot performs a short waiting process to observe its surroundings and allow potential interaction opportunities.

3.4.5.8 Goal-Directed Navigation State

When a target location is provided, the system transitions into the goal-directed navigation state, where it executes path planning and motion control to reach the specified destination. The robot continuously adjusts its movement based on localisation updates and environmental feedback to ensure accurate and safe navigation. In this project, the target goal is not generated autonomously but is obtained from the human interaction process, where the selected destination corresponds to predefined goal points already marked within the navigation map.

3.4.5.9 HTTP Bridge for External System Integration

The pi5_service node provides a REST API using FastAPI to connect the AI server with the ROS2 system.

The service listens on port 8767 and exposes two endpoints that correspond to the main interaction events defined in the system design. The POST /nav_state endpoint accepts a state integer and an optional destination codename, and the GET /health endpoint returns the current navigation state.

The node is launched alongside the NavigationManager using a single launch file, with ROS2 spinning in a background thread while the FastAPI server runs in the main thread. Upon receiving a navigation request, the service validates the state value and, for goal-directed commands, performs a dictionary lookup to translate the room codename to the corresponding ROS2 goal name. The translated command is then

published as a string message on /navigation_command, which is consumed by the navigation state machine.

3.5 SOFTWARE DESIGN AND DEVELOPMENT

3.5.1 Software Architecture Overview

The system is structured into three tightly integrated layers: (1) a vector database for semantic retrieval, (2) a relational database for structured data management, and (3) an LLM integration module for conversational intelligence. These components work together to support real-time interaction, contextual reasoning, and personalised responses.

The five software repositories that constitute the system are assigned to their respective hardware nodes as follows.

- JetsonVision – Python application deployed on the NVIDIA Jetson Orin Nano. Responsible for real-time face detection, multi-face tracking, head pose estimation, and face recognition.
- RaspiReceive – Python asyncio application deployed on Pi 5 #1. Responsible for microphone capture, voice activity detection, speech transcription, detection-speech correlation, and audio playback.
- baymax-face-ui – Electron.js frontend combined with a Python FastAPI backend, deployed on Pi 5 #1. Renders the robot's emotional face on an HDMI display.
- capstone_server – Python FastAPI application deployed on the AI Brain server. Hosts the complete AI pipeline including grammar correction, RAG retrieval, LLM inference, TTS synthesis, and response dispatch.
- Ros2_robot – ROS2 Jazzy package suite deployed on Pi 5 #2. Provides hardware bring-up, sensor drivers, AMCL localisation, Nav2 path planning, and an HTTP bridge for external command injection.

3.5.2 Database Design

A hybrid database architecture is used to combine the advantages of both vector-based and relational storage systems. The vector database supports efficient semantic retrieval of unstructured data, while the relational database maintains data integrity and enables reliable transactional processing.

3.5.2.1 Milvus Vector Database

Milvus functions as the system's semantic memory layer, storing vector embeddings of unstructured data such as university information, curriculum materials, and timetable data. Instead of storing raw text, documents are transformed into 1024-dimensional dense vectors using the BAAI/bge-m3 model, enabling efficient similarity-based retrieval.

Each collection is indexed using the IVF_FLAT method with nlist set to 128. During querying, an nprobe value of 10 is used, allowing approximate nearest-neighbor (ANN) search based on cosine similarity. The database is organised into four primary collections:

- uni_info: Contains seven entries describing campus buildings, facilities, zones, and departmental information within Building E-12 at KMITL. Serves as the default fallback retrieval source for campus navigation queries.
- curriculum: Derived from the Robotics and Artificial Intelligence Engineering programme specification document, containing 516 Thai-language chunk embeddings covering course descriptions, learning outcomes, prerequisites, and credit structures for Years 1 through 4.
- time_table: Constructed from an academic Excel timetable dataset. Each entry encodes a structured Thai sentence of the form (day, time slot, course name, room) to maximise retrieval precision, resulting in 128 entries.
- local_info: Contains information on nearby bars and restaurants around the campus. Each entry encodes the place name, category, location, opening hours, and a brief description to support lifestyle and dining-related queries

from students.

- conversation_memory: A dynamic collection storing per-student LLM-generated conversation summaries, indexed by student_id. Supports filtered semantic retrieval, allowing the system to recall contextually relevant past interactions for a specific student during personalised prompt construction.

Semantic retrieval is performed using approximate nearest-neighbour search with an IVF_FLAT index and cosine similarity, significantly reducing response latency compared to traditional search methods.

3.5.2.2 MySQL Relational Database

MySQL manages structured data required for system operations. The schema enforces referential integrity and supports efficient querying of user-related information.

Key tables include:

- Academic_Year: Stores academic calendar information, including year boundaries used for tone adaptation in personalised prompts.
- Students: Maintains student identity records, including student identifier, full name and nickname in Thai and English, academic year, and enrolment status. This table is the authoritative source for student metadata injected into LLM prompts.
- Face_Recognition_Data: Stores biometric face encodings linked to student identities. Used by the Jetson Orin Nano enrolment tool to maintain the face embedding database.
- ExcelTimetableData: Contains structured timetable records for precise SQL-based schedule lookups, complementing the semantic retrieval provided by the time_table Milvus collection.

3.5.2.3 Database Synchronisation

Timetable data is synchronised across both systems: MySQL stores structured records, while Milvus stores corresponding embeddings for semantic retrieval. Updates are applied to both layers to maintain consistency between structured queries and

conversational responses.

The conversation_memory collection is updated asynchronously at the end of each conversation session. The 8B LLM summarises the session transcript into a concise natural-language paragraph, which is then embedded and stored in Milvus tagged with the student's student_id. This fire-and-forget pattern prevents memory writes from blocking the main response pipeline.

3.5.3 LLM Integration Module

The LLM module serves as the conversational core, implementing a multi-route RAG pipeline that dynamically selects the appropriate data source for each query. The system uses a dual-model setup: a high-capacity 70B model for response generation and a lightweight 8B model for preprocessing tasks such as grammar correction and memory summarisation. Both models run via the Ollama inference server with GPU acceleration.

- Main chatbot: llama3.1-typhoon2-70b-instruct (Q5_K_M GGUF) – used for greeting generation and final response synthesis.
- Fast assistant: llama3.1-typhoon2-8b-instruct – used for grammar correction and end-of-session memory summarisation.
- Audio sidecar: typhoon-ai/llama3.1-typhoon2-audio-8b-instruct – a multimodal model deployed as an independent sidecar process on port 8001, handling both STT transcription and TTS synthesis.

3.5.3.1 Query Routing Strategy

User queries are classified using keyword-based intent detection and routed to one of six data sources:

- Conversation Memory: Retrieves the top-3 most semantically similar past conversation summaries from the conversation_memory Milvus collection, filtered by student_id. Provides long-term personalisation context.
- Student Information: Executes a structured SQL query against MySQL to retrieve the student's academic year and metadata. The student_year field

defaults to 1 on failure to prevent pipeline interruption.

- Timetable: Activated when keywords associated with scheduling are detected in the query (e.g., day, time slot, timetable). Retrieves top-3 embeddings from time_table and optionally augments with a MySQL lookup for precise room and time data.
- Curriculum: Activated when course-related keywords are present (e.g., subject name, credit, course code). Performs semantic search over the curriculum collection (top-3 chunks) to retrieve programme-specific information.
- Local Information: Activated when keywords related to nearby dining and lifestyle are detected in the query (e.g., restaurant, food, drink). Retrieves top-3 embeddings from the local_info collection to provide recommendations on bars and restaurants located near the campus.
- University Information: Default fallback channel. Retrieves from uni_info when no other channel matches, covering campus layout, building zones, and general facilities within Building E-12 at KMITL.
- General Chat: Handles open-ended conversational queries without external retrieval. The LLM generates responses solely from its parametric knowledge and the current session history.

Figure 3.18 illustrates how the keyword classifier routes each incoming query through the appropriate retrieval channel before LLM generation.

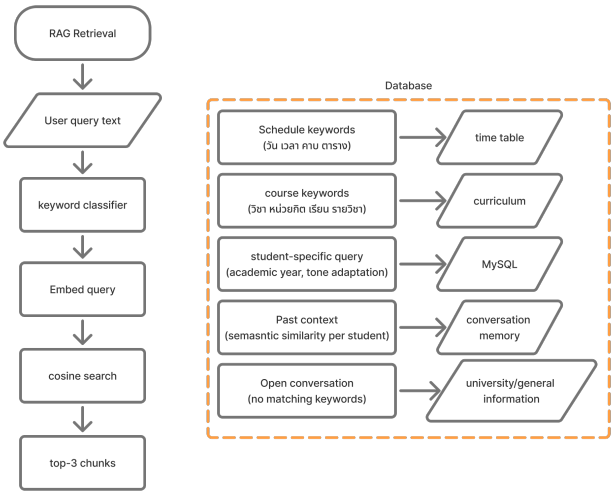


Figure 3.18 RAG Query Routing: Keyword Classifier Directing Queries to Six Retrieval Channels

Table 3.5 Milvus Vector Database Collections and Retrieval Signals

Collection	Size	Content	Query Signal
curriculum	516 chunks	Course descriptions, learning outcomes, prerequisites, and credit structures for Years 1–4 (RAI programme)	Course name and subject code keywords
time_table	128 entries	Structured Thai sentences encoding (day, time slot, course name, room) tuples	Schedule-related keywords
uni_info	7 entries	Campus buildings, facilities, and zones within Building E-12 at KMITL	Building and zone queries
conversation_memory	Dynamic	LLM-summarised past session transcripts, tagged per student_id	Semantic similarity + student_id filter

3.5.3.2 Personalised Prompt Generation

Responses are dynamically tailored using student metadata, including name and academic year. Tone adaptation is applied based on student level, ensuring appropriate communication style. Prompts also include recent conversation summaries and retrieved contextual knowledge to maintain coherence.

The system prompt specifies the robot's persona as a female Thai-language assistant with the following behavioural constraints:

- All speech particles must be feminine; masculine particles and first-person masculine pronouns are prohibited.
- The student is addressed by nickname only; full names are never used mid-conversation.
- Responses are capped at two sentences to ensure TTS output remains concise and intelligible.
- Numbers, acronyms, and building codes are expanded into their full Thai phonetic representations to ensure correct TTS pronunciation (e.g., E-12 is rendered as its Thai phonetic equivalent; room numbers are spoken digit-by-digit).

3.5.3.3 Output Schema and Intent Classification

The LLM returns structured JSON outputs containing response text and intent classification. Supported intents include:

- chat / info: Triggers speech output only.
- navigate: Triggers speech and ROS2 navigation commands.
- farewell: Ends session and resumes autonomous roaming.

3.5.3.4 Session Memory and Persistence

The system uses a dual storage strategy:

- SQLite: Stores full conversation logs for auditing and debugging.
- Milvus: Stores summarised embeddings for long-term semantic memory.

Session states are maintained in-memory with timeout policies (30 minutes for registered users, 5 minutes for guests). A silence-based timeout mechanism triggers re-prompting and automatic session termination. A per-student greeting cooldown of 300 seconds is enforced by the /greeting endpoint to prevent repeated activation caused by transient face detection events within a single encounter.

3.5.4 Speech Processing Subsystem

The speech processing subsystem spans three tightly coupled components: the Speech-to-Text and Grammar Correction module, the Text-to-Speech module, and the Audio Sidecar Service. Together they handle all audio input and output across the interaction pipeline.

3.5.4.1 Speech-to-Text and Grammar Correction

Speech processing is divided into two stages: transcription and linguistic refinement.

ASR via Typhoon2-Audio:

Audio input is sent to the Typhoon2-Audio sidecar via a POST /stt request. The model processes 16 kHz PCM audio and returns Thai transcription with typical latency of 1–3 seconds.

The system supports two operational STT modes. Mode B is the recommended production configuration as it leverages GPU-accelerated inference on the server and eliminates local model loading on the Pi.

- Mode A (edge STT): Pi 5 #1 performs on-device transcription and sends the resulting text as a JSON payload to the server via /detection. Two backends are available under Mode A. The recommended backend is scb10x/typhoon-asr-realtime via the NVIDIA NeMo Toolkit (–stt-backend typhoon-local), achieving 0.68–2.0 seconds per segment on Pi 5 CPU with no hallucinations observed in live testing. The fallback backend is biodatlab/distill-whisper-th-small via faster-whisper INT8 quantisation (–stt-backend whisper), achieving 8.5–9.7 seconds per segment and requiring hallucination mitigations includ-

ing VAD filtering, average log-probability thresholding, and a max_new_tokens cap of 50.

- Mode B (server STT): Pi 5 #1 forwards raw PCM WAV audio as a multipart payload to /audio_detection. The server invokes the Typhoon2-Audio-8B sidecar for GPU-accelerated transcription before the remaining inference pipeline proceeds. This mode requires no local STT model on the Pi, enabling startup in approximately 3 seconds.

Table 3.6 STT Backend Comparison for Edge Audio Coordination Subsystem

Backend	Speed (Pi 5 CPU)	RAM	Role
scb10x/typhoon-asr-realtime (NeMo)	0.68–2.0 s/seg	~2.8 GB + 1.7 GB swap	Mode A primary
biodatlab/distill-whisper-th-small (faster-whisper INT8)	8.5–9.7 s/seg	~800 MB–1 GB	Mode A fallback
Typhoon2-Audio-8B (server GPU)	1–5 s/seg	GPU (server)	Mode B (recommended)

3.5.5 Grammar Normalisation

The 8B LLM corrects spelling and phonetic errors while preserving meaning. Strict constraints ensure that no additional content is introduced. Grammar correction is bypassed when the STT confidence score exceeds a threshold of 0.85, avoiding unnecessary LLM invocation for high-confidence transcriptions.

3.5.6 Robustness Mechanisms

Several safeguards are implemented:

- Minimum input length filtering – transcriptions shorter than three characters are rejected by a noise gate and do not trigger a server inference call.
- Output length validation (50–150%) – the grammar-corrected output must

fall within 50–150% of the input character length. Outputs outside this range trigger reversion to the original raw transcription.

- Timeout fallback to raw transcription – if the grammar correction LLM call exceeds a configurable timeout, the raw transcription is used without correction to maintain conversational responsiveness.

3.5.6.1 Text-to-Speech Module

The TTS subsystem converts responses into spoken Thai using a preprocessing and synthesis pipeline. It handles Thai phonetic structures and converts non-Thai elements into readable forms.

Text is sent to the Typhoon2-Audio sidecar via /tts, where it is synthesised into 16 kHz PCM WAV audio. The output is then forwarded to Raspberry Pi 5 for playback. The sidecar is preferred due to its faster GPU-based inference compared to edge-only processing.

Text Normalisation Preprocessing includes:

- English and acronym expansion.
- Number-to-Thai conversion.
- Unicode normalisation.
- Word and syllable segmentation.

Asynchronous Dispatch TTS and navigation commands are executed asynchronously using asyncio, ensuring the main pipeline remains non-blocking and responsive. This pattern eliminates the 15–25 second blocking observed when TTS was awaited synchronously on slow Raspberry Pi 5 networks, freeing the server to accept the next student interaction immediately.

3.5.6.2 Audio Sidecar Service

The Audio Sidecar is an independent FastAPI process running on the AI Brain server on port 8001. It is isolated in a separate Python 3.10 virtual environment to avoid model dependency conflicts with capstone_server. The service is backed by the

typhoon-ai/llama3.1-typhoon2-audio-8b-instruct multimodal model, loaded once at server startup in float16 precision with attn_implementation="eager". Flash-attention is intentionally disabled because GLIBC_2.32 is unavailable on the Ubuntu 20.04 host environment. The model is shared across all requests via a singleton pattern to eliminate repeated load overhead.

Figure 3.19 illustrates the two independent processing flows—TTS synthesis and STT transcription—exposed through the sidecar's endpoints.

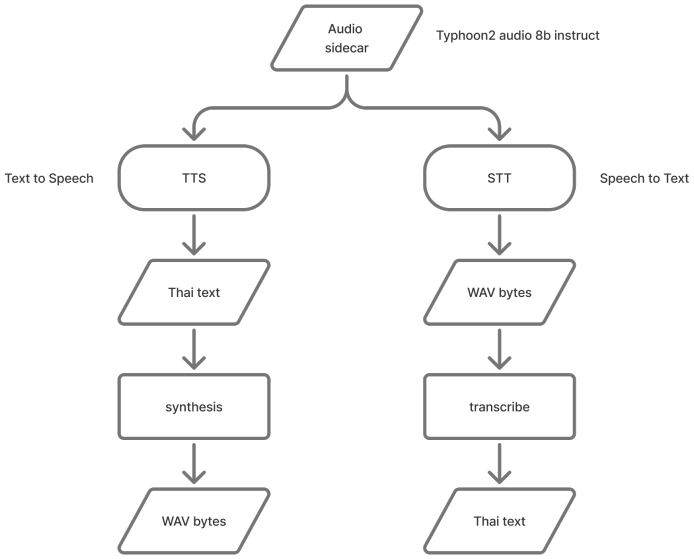


Figure 3.19 Audio Sidecar Service: TTS Synthesis and STT Transcription Processing Flows (Port 8001)

1. TTS Synthesis Flow: The sidecar receives a Thai text string from capstone_server via POST /tts {"text": "..."}. It invokes model.synthesize_speech(text), which returns a float32 NumPy audio array at a 16 kHz sampling rate. The array is encoded to a BytesIO buffer using soundfile in WAV format with PCM_16 subtype, and returned as raw WAV bytes with Content-Type: audio/wav. Typical output is approximately 105 KB per utterance with a synthesis latency of approximately 633 ms on GPU.

2. STT Transcription FlowThe sidecar receives raw WAV bytes in the HTTP request body via POST /stt. Because the model API requires a file path rather than an in-memory buffer, the bytes are first written to a temporary file. A multi-turn conversation is then constructed with a system instruction and a user message containing an audio content block pointing to the temporary file. The model generates a transcription with parameters max_new_tokens=256, do_sample=False, and temperature=1.0. The temporary file is deleted in a finally block regardless of success or failure. Typical transcription latency is 1–5 seconds on the GPU server.

Table 3.7 Audio Sidecar API Endpoints (Port 8001)

Method	Path	Input	Output
POST	/tts	{”text”: str}	WAV bytes (audio/wav)
POST	/stt	Raw WAV bytes body	{”text”: str}
GET	/health	—	{”status”: ”ok”, ”model”: model_id}

3.5.7 Inference Pipeline

Each request to /detection (text payload from edge STT) or /audio_detection (raw WAV with metadata from Mode B) triggers a sequential eight-stage pipeline on the AI Brain server. Figure 3.20 provides a stage-level overview of the complete flow from input filtering to asynchronous response dispatch.

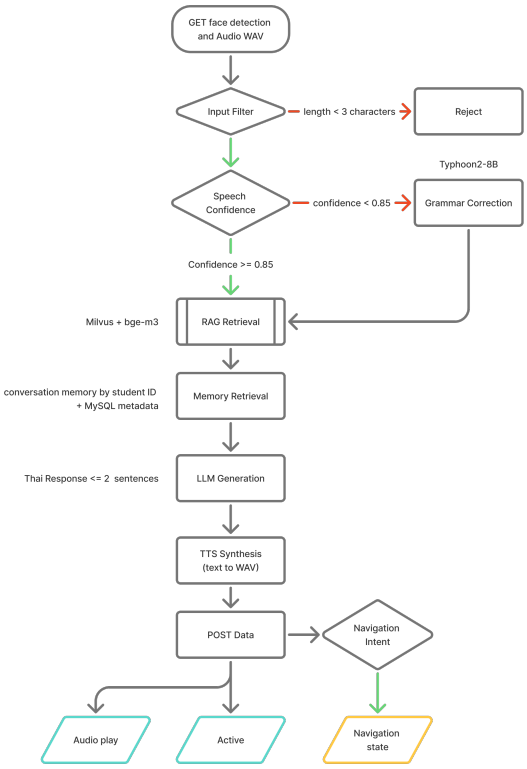


Figure 3.20 AI Inference Pipeline: Eight Sequential Processing Stages from Input Filter to Asynchronous Dispatch

1. Input Filter (Noise Gate). Transcriptions shorter than three characters are rejected without triggering any downstream model inference. This eliminates spurious activations from coughs, brief sounds, or single-character STT outputs caused by background noise.
2. Grammar Correction. The lightweight Typhoon2-8B model corrects spelling and phonetic errors introduced by ASR (e.g., mishearing Thai tone marks or vowel clusters). Correction is bypassed entirely when the STT confidence score exceeds 0.85, avoiding an unnecessary LLM call for high-quality transcriptions. A length guard ensures the corrected output falls within 50–150% of the input character length; outputs outside this range revert to the original raw transcription. A timeout fallback also reverts to raw transcription if the correction call exceeds the configured limit.

3. Query Routing and Context Retrieval. A keyword-based classifier maps the query to one of six retrieval channels (conversation_memory, student_info, timetable, curriculum, uni_info, or general_chat), determining which Milvus collection or MySQL table is queried in the next stage. The top-3 most semantically relevant chunks are retrieved using BAAI/bge-m3 embeddings and cosine similarity search.
4. Memory Retrieval. For registered students, the top-3 most semantically similar past conversation summaries are retrieved from the conversation_memory Milvus collection, filtered by student_id. A MySQL query also retrieves the student's academic year and nickname for personalised prompt tone and address style.
5. LLM Response Generation. The Typhoon2-70B model (Q5_K_M GGUF via Ollama) generates a Thai-language response. The system prompt enforces the robot's persona: feminine speech particles, nickname-only address, a maximum of two sentences, and phonetic expansion of building codes and numbers for TTS clarity (e.g., building code E-12 is expanded to its Thai phonetic equivalent; room numbers are spoken digit by digit). The model also classifies the response intent in structured JSON output.
6. Asynchronous Logging and Memory Update. Session history is updated after each exchange. An end-of-session memory summarisation task is enqueued asynchronously so that memory writes do not block the main response pipeline.
7. Intent-based Dispatch (TTS / Navigation). The response text undergoes preprocessing—acronym expansion, number-to-Thai conversion, Unicode normalisation, and word segmentation—before being sent to the Audio Sidecar via POST /tts. All downstream commands (WAV delivery to Pi 5 #1 at /audio_play, microphone activation flag at /set_active, and navigation commands to Pi 5 #2 at /nav_state when intent is navigate) are fired concurrently using asyncio, eliminating the 15–25 second serial delay observed when these calls were

awaited synchronously.

8. System Reactivation. The microphone activation flag (/set_active active=1) is sent after audio playback completes, not immediately, ensuring the robot cannot transcribe its own synthesised speech output.

Table 3.8 AI Inference Pipeline Latency Summary (Single GPU Server)

Stage	Model / Component	Typical Latency
Grammar Correction	Typhoon2-8B	800–1,500 ms
RAG Retrieval	Milvus + bge-m3	50–200 ms
Memory Retrieval	Milvus + MySQL	30–80 ms
LLM Generation	Typhoon2-70B	4,000–12,000 ms
TTS Synthesis	Typhoon2-Audio-8B	~633 ms
Network (Pi 5 Transfer)	WAV Delivery	~50 ms
Total End-to-End	—	6–15 s per turn

3.5.8 Vision and Face Recognition Subsystem

The vision subsystem (JetsonVision) is the first processing stage in the interaction pipeline. It operates continuously on the Jetson Orin Nano and is responsible for detecting, tracking, and identifying human faces in the robot's camera field of view. The processing stages—from raw camera frame to WebSocket event dispatch—are illustrated in Figure 3.21.

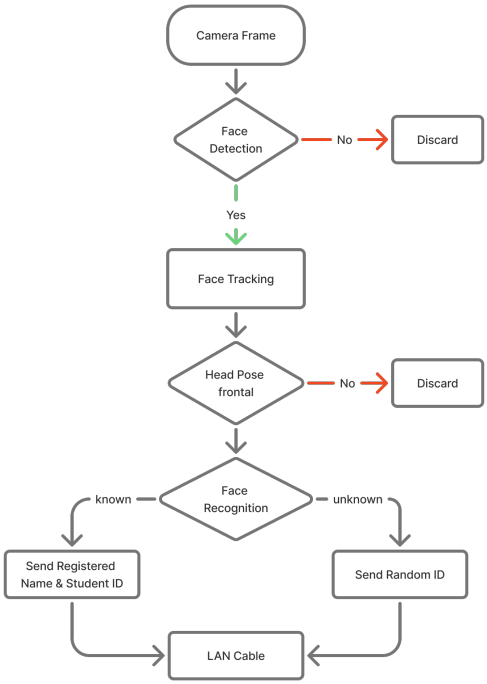


Figure 3.21 JetsonVision Pipeline: Face Detection, Multi-Object Tracking, Head Pose Gating, and Recognition

3.5.8.1 Detection and Tracking

Face detection is performed using the SCRFD (Sample and Computation Re-distribution Face Detector) model with a 640×640 input resolution. The detector is configured with a confidence threshold of 0.5 and a Non-Maximum Suppression (NMS) IoU threshold of 0.4. The model is accelerated using TensorRT via the ONNX Runtime TensorRT execution provider when operating in GPU mode; a CPU-only ONNX fallback is available via the –cpu-only command-line flag.

SCRFD produces bounding boxes and five facial landmark points per detected face. These outputs are passed to a ByteTracker multi-object tracker, which assigns a persistent track_id to each face across consecutive frames. ByteTracker maintains track continuity through brief occlusions and frames in which detection confidence drops below the threshold, providing stable identity anchors for downstream recognition.

3.5.8.2 Head Pose Estimation and Frontal Gate

A lightweight head pose estimator computes the yaw, pitch, and roll of each tracked face from the five SCRFD landmark points. Only faces satisfying the frontal constraint of yaw less than 25 degrees and pitch less than 15 degrees are eligible for recognition. This gate prevents recognition attempts on profile views, where ArcFace embedding quality degrades significantly, thereby reducing false-positive matches.

3.5.8.3 Face Recognition

Eligible face crops are passed to an ArcFace model with a ResNet backbone that produces 512-dimensional face embeddings. Inference is performed in FP16 precision to utilise the Tensor Cores available on the Jetson Orin Nano's GPU. A cooldown gate of 5 seconds per track_id ensures that repeated recognition calls for the same continuously-present face do not saturate the system.

Recognition is performed by computing the cosine similarity between the query embedding and each stored embedding in data/enrollments.json. If the maximum similarity across all enrolment entries for a given identity exceeds a configurable threshold, the face is labelled as a known student and the associated metadata is returned; otherwise, the face is labelled unknown and assigned a temporary UUID as its person_id.

Student enrolment is performed offline using the enroll_student.py tool, which guides the operator to capture five face angles (frontal, left, right, and two 45-degree intermediate poses) and stores the corresponding embeddings alongside the student's Thai and English full names and nickname.

3.5.8.4 WebSocket Event Emission

Upon a successful recognition or unknown-face detection event, the Jetson transmits a JSON message to Pi 5 #1 over the persistent WebSocket connection on port 8765. The message schema carries a top-level status field set to detected, and a nested metadata object containing the fields eng_name, thai_name, student_id, is_registered, and confidence. For unrecognised faces, is_registered is set to false and the name fields are null.

On the receiving side, the Pi 5 VisionReceiver applies a confidence threshold of 0.75 before forwarding detection events to the coordinator. A 15-second debounce is applied per face, ensuring that the detection callback fires at most once every 15 seconds for a continuously present individual, preventing the asyncio queue from being flooded during the STT processing window. Null identity fields are mapped to the string value Unknown before use in downstream payloads.

The WebSocket message emitted on each detection or recognition event carries the following JSON structure:

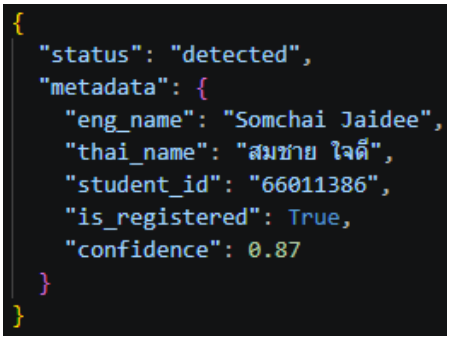


Figure 3.22 Websocket message

For unrecognised faces, is_registered is set to false, the name fields are null, and a temporary UUID is assigned as the person_id in the downstream payload.

3.5.9 Edge Audio Coordination Subsystem

The edge audio coordination subsystem (RaspiReceive) deployed on Pi 5 #1 serves as the interaction hub. It bridges the vision subsystem and the AI server by combining face detection events with speech transcription results into a unified payload, while managing audio capture and playback. The complete audio coordination flow—from microphone capture through VAD, STT, combiner gating, and HTTP dispatch—is shown in Figure 3.23.

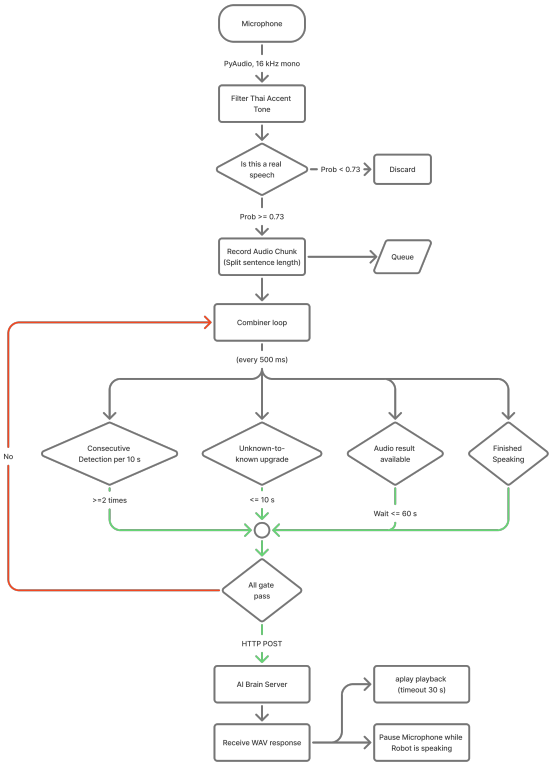


Figure 3.23 RaspiReceive Pipeline: Pre-emphasis, Voice Activity Detection, STT, Combiner Gating, and Server Dispatch

3.5.9.1 Concurrent Task Architecture

RaspiReceive is implemented as a single asyncio event loop running four concurrent coroutines via asyncio.gather:

- _detection_loop(): Listens on WebSocket port 8765 for incoming Jetson events, applies session upgrade logic from Unknown to Known, and enqueues events for the combiner.
- _stt_consumer_loop(): Drains the output queue of the AudioWorker background thread, accumulating transcription results in a rolling recent buffer.
- _combiner_loop(): Executes every 500 ms, matching detection events with transcription results according to the gating rules described below, and dispatching combined payloads to the AI server.
- _run_activation_server(): Hosts the FastAPI HTTP service on port 8766, provid-

ing endpoints for audio playback, microphone activation control, and health monitoring.

A background AudioWorker thread, running outside the asyncio event loop, handles blocking microphone input/output via PyAudio at 16 kHz mono. Captured audio frames are accumulated in a three-second rolling circular buffer from which speech segments are extracted by the SileroVAD model. The AudioWorker is paused during TTS playback and resumed upon completion, preventing the robot from transcribing its own synthesised speech output.

The activation state is managed through a push-based model: the server posts /set_active with active = 1 to re-enable the microphone after each response, and active = 0 to suppress new input during processing. An auto-reactivation timer of 60 seconds serves as a fallback in the event that the server call does not arrive.

3.5.9.2 Voice Activity Detection and Transcription

A two-tier VAD pipeline is employed to eliminate hallucinations caused by silent or low-energy ambient audio. In the first tier, a pre-emphasis filter boosts the amplitude of Thai tonal consonants before analysis. In the second tier, the SileroVAD model assigns a speech probability to each audio chunk; chunks with probability exceeding the threshold of 0.73 are classified as speech. Only frames passing both tiers are appended to the utterance accumulator. This two-tier approach eliminates the hallucination loops observed with single-stage VAD, where models such as faster-whisper would generate repeated Thai or English phrases on background noise.

Upon detecting the end of a speech segment, the accumulated audio is submitted to the active STT backend. The transcription result is wrapped in an STTChunk dataclass and placed on the output queue consumed by _stt_consumer_loop(). The microphone input is automatically muted during TTS audio playback by pausing the AudioWorker, preventing the robot from transcribing its own synthesised speech. The AudioWorker resumes microphone capture only after playback has completed.

3.5.9.3 Detection-Speech Combiner Logic

The _combiner_loop() implements four sequential gating rules before dispatching a payload to the AI server:

1. Consecutive detection gate: At least two detection events for the same track_id must arrive within a ten-second sliding window before a send is triggered. This prevents false activations from brief, transient face detections.
2. Unknown-to-Known upgrade window: When an unknown face is first detected, the combiner holds the event for up to ten seconds (unknown_hold_sec = 10) waiting for the same track_id to receive a known identity from a subsequent Jetson recognition event.
3. STT wait: After a valid detection event passes the consecutive gate, the combiner waits up to 60 seconds (stt_wait_sec = 60) for a non-empty transcription result. If the timeout expires, the payload is dispatched with an empty text field, triggering a contextual greeting on the server side.
4. Post-send cooldown: After dispatching a payload, the combiner ignores new detection events for five seconds (cooldown_sec = 5) to prevent duplicate triggers within the same interaction.

Table 3.9 RaspiReceive Subsystem Configuration Parameters

Parameter	Default	Description
websocket_port	8765	Jetson detection listener port
audio_vad_threshold	0.73	SileroVAD speech probability gate
stt_model_name	distill-whisper-th-small	Whisper model ID
cooldown_sec	5.0	Post-send detection cooldown (s)
unknown_hold_sec	10.0	Unknown-to-Known upgrade window (s)
stt_wait_sec	60.0	Maximum wait for STT result (s)
consecutive_detections_required	2	Detections needed within window
activate_timeout_sec	60.0	Auto-reactivate if server silent (s)
max_tts_duration_sec	30.0	Playback watchdog timeout (s)

3.5.9.4 Audio Playback and Watchdog

Synthesised audio (WAV) received at the /audio_play endpoint is queued for sequential playback via the system's aplay command. A background watchdog task monitors each aplay subprocess and force-terminates it if playback has not completed within 30 seconds (max_tts_duration_sec = 30). This safeguard prevents the audio pipeline from entering a blocked state due to network transfer errors or corrupted WAV data.

3.5.10 Expressive Robot Interface Subsystem

The expressive interface subsystem (baymax-face-ui) renders an animated robot face on the HDMI display connected to Pi 5 #1. The system communicates the robot's internal state to the interacting student through five discrete emotional expressions that are mapped to interaction phases.

3.5.10.1 Expression System

The face is rendered on an HTML5 Canvas element within a fullscreen Electron.js BrowserWindow. Animation is driven by a requestAnimationFrame loop with interpolated transitions between expression states. Table 3.4 describes the five supported expressions, and Figure 3.24 illustrates the complete state transition sequence across a full interaction cycle, from person detection through response playback and navigation.

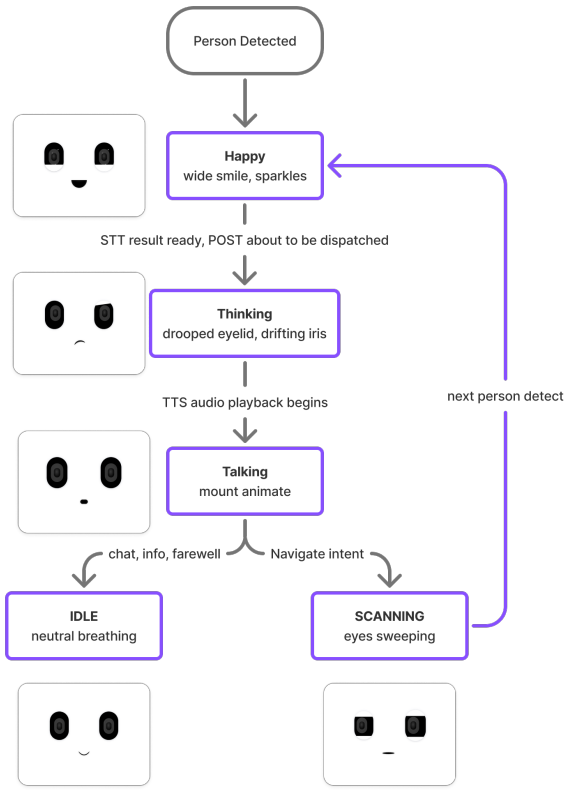


Figure 3.24 Robot Face Expression Flow: State Transitions Driven by Interaction Lifecycle Events

Table 3.10 Robot Face Expression States

Code	Name	Visual Description	Triggered By
0	Idle	Neutral with slow breathing animation	Playback ends (chat / info / farewell)
1	Scanning	Eyes sweep left-right	Navigating to a destination
2	Happy	Wide smile, sparkle particles	Person detected, greeting about to fire
3	Talking	Mouth animates vowel shapes	TTS audio playback in progress
4	Thinking	Drooped eyelid, drifting iris	STT ready, awaiting LLM response

3.5.10.2 IPC Architecture

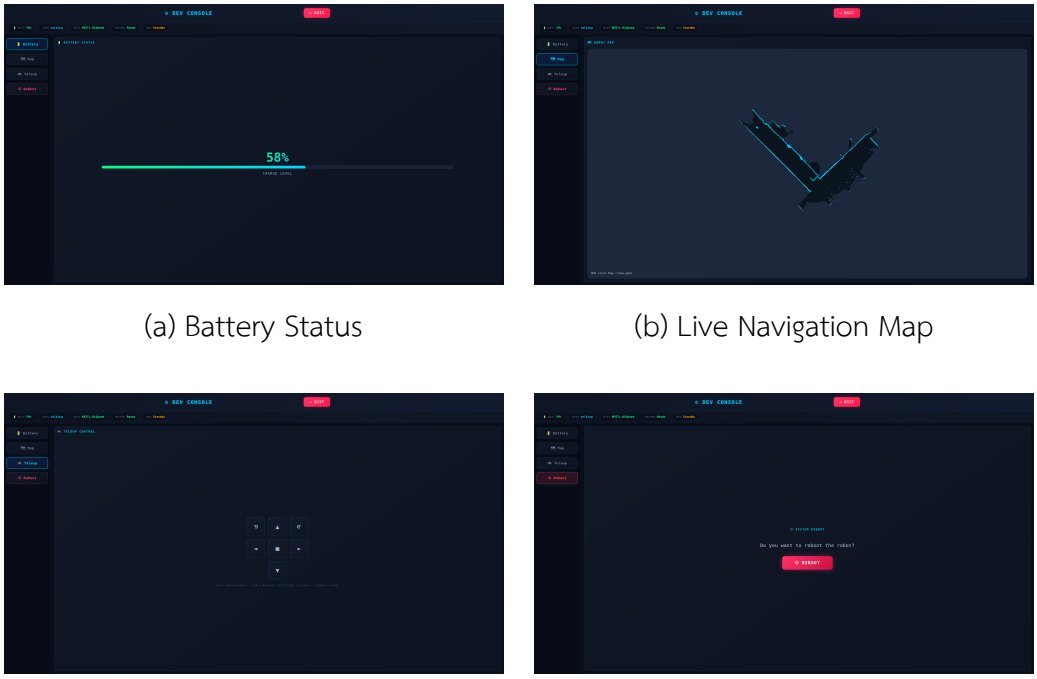
All face emotion transitions are managed locally by Pi 5 #1. The server does not call the face service directly. Instead, the raspi_main.py coordinator drives all

transitions based on its own visibility into the interaction lifecycle: it sets happy (2) as soon as a person is identified and before posting to /greeting; it sets thinking (4) when the STT result is ready and the POST to the server is about to be dispatched; it sets talking (3) when audio playback begins; and it sets idle (0) when playback ends. For navigation intents, the transition after playback ends is scanning (1) rather than idle (0), persisting until the next interaction.

Face calls are issued as fire-and-forget HTTP requests so that a slow or unresponsive face service never blocks the audio delivery pipeline. A Python FastAPI service (pi5_face_service.py) listens on port 7000 and accepts HTTP POST requests at the /face_emotion endpoint. The request body specifies the target emotion code as a JSON integer field. The FastAPI process forwards emotion codes to the Electron renderer process via an internal WebSocket channel on localhost:8768, which acts as an IPC bridge. The Electron main process listens on this channel and invokes the appropriate canvas state transition in the renderer. This two-process architecture decouples the HTTP-facing service from the rendering engine, enabling independent restart of either component without disrupting the other.

3.5.10.3 Developer Interface Mode

In addition to the standard animated face display, the interface subsystem provides a developer mode overlay intended for diagnostics and manual control during deployment and maintenance. The overlay replaces the animated face canvas and exposes four dedicated panels, each accessible via the developer mode toggle in the Electron application.



(a) Battery Status (b) Live Navigation Map

(c) Teleoperation Control (d) System Reboot

Figure 3.25 Developer Interface Mode: Four Diagnostic Panels Accessible via the Electron Overlay

The Battery Status panel displays real-time voltage and estimated charge level, allowing operators to assess power availability without accessing the hardware directly. The Live Navigation Map panel renders the AMCL-estimated robot pose overlaid on the occupancy grid map, providing immediate visual confirmation of localisation quality. The Teleoperation Control panel exposes directional movement buttons that publish velocity commands directly to the ROS2 /cmd_vel topic, enabling manual repositioning when autonomous navigation is not active. The System Reboot panel provides a one-tap interface to gracefully restart the robot's software stack, including both the RaspiReceive process and the face display service, without requiring SSH access.

3.5.11 Autonomous Navigation Subsystem

The navigation subsystem (Ros2_robot) deployed on Pi 5 #2 provides autonomous mobile navigation within the E12 Building floor at KMITL. It integrates hardware drivers, a LiDAR-based localisation stack, and a path planning framework, exposed to external control via an HTTP bridge.

3.5.11.1 Hardware Layer and Odometry

The robot chassis is modelled in URDF via the ired.urdf.xacro file, which defines the base_link, base_footprint, left and right drive wheel joints, IMU frame, and RPLiDAR mounting point. The iREDCr differential drive controller communicates with the wheel motors over a serial interface at /dev/ttyiREDCr and publishes wheel odometry to the /odom_encoder topic.

A supplementary odometry source is provided by a scan-to-scan laser scan matcher (ros2_laser_scan_matcher), which publishes its estimate to the /odom topic. The scan matcher is configured with keyframe thresholds of 0.05 m and 0.087 rad (approximately 5 degrees), triggering a new scan match only when the robot has moved or rotated beyond these limits. This laser-derived odometry exhibits lower drift under wheel slippage conditions and is used as the primary input to the AMCL particle filter.

3.5.11.2 Localisation and Path Planning

Localisation is performed by Adaptive Monte Carlo Localisation (AMCL) against a pre-built 2D occupancy grid map of the operating environment. The particle filter is seeded from an initial pose estimate; when no initial pose is available at startup, the robot executes a 360-degree in-place rotation to gather sufficient scan data for the filter to converge.

Path planning and execution are managed by the Nav2 stack. The global planner generates a collision-free path from the current estimated pose to the goal pose, and the DWB (Dynamic Window B) local controller tracks this path. The DWB controller is configured with a maximum translational velocity of 0.3 m/s and a stopped-velocity threshold of 0.25 m/s. The stopped-velocity threshold must be set strictly less than the maximum velocity to prevent the controller from interpreting normal cruising speed as a stopped condition, which was found during development testing to cause an indefinite in-place spinning behaviour.

3.5.11.3 HTTP Bridge and Navigation State Machine

An HTTP bridge service (pi5_service.py), deployed as a standalone Python FastAPI application on port 8767, translates external HTTP commands from the AI Brain server into ROS2 navigation goals. The bridge exposes a /nav_state endpoint that accepts a JSON body containing the fields state (integer 0, 1, or 2) and destination (room name string). The full navigation state machine, including goal dispatch, stuck-detection sub-sequence, and retry logic, is illustrated in Figure 3.26.

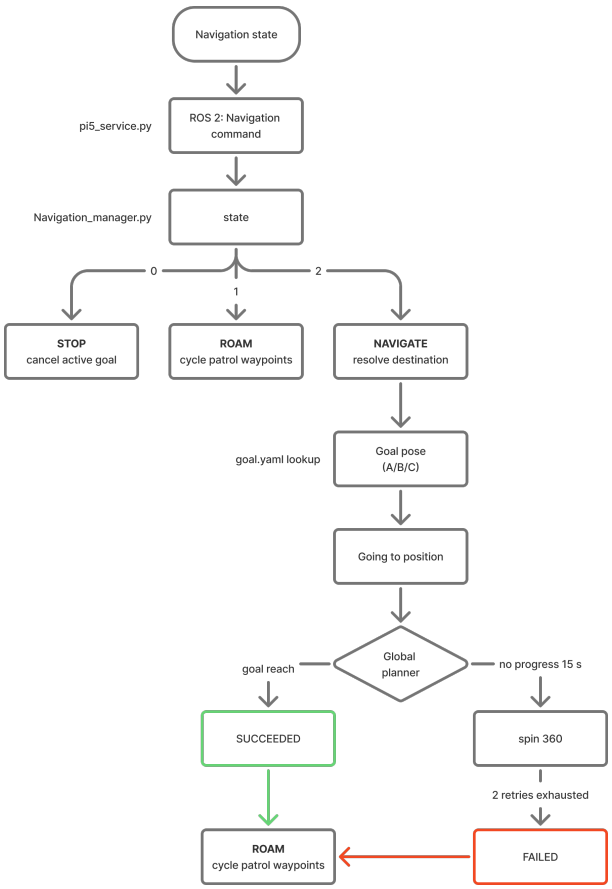


Figure 3.26 Autonomous Navigation State Machine: Goal Dispatch, Stuck Detection (360° Recovery), and Retry Logic

Destination room names are resolved to metric goal poses (x, y, yaw in the map frame) via a static mapping defined in goals.yaml. The room-to-codename mapping is

Table 3.11 Navigation State Definitions

State	Label	Action
0	Stop	Cancel active navigation goal; robot halts in place
1	Roam	Cycle through patrol waypoints continuously
2	Navigate	Execute goal-directed navigation to a named room

as follows: e12-1 maps to label A, e12-2 maps to label B, and e12-3 maps to label C. Goal poses are determined experimentally by driving the robot to each target location and recording the AMCL pose estimate from the /amcl_pose topic.

The navigation_manager.py ROS2 node wraps the Nav2 BasicNavigator API and publishes the current navigation status to the /navigation_status topic with values IDLE, ROAMING, NAVIGATING, SUCCEEDED, or FAILED. The node implements a stuck-detection mechanism: if the robot's displacement over a 15-second window is less than 0.05 m, it is considered stuck, and the node commands a 360-degree in-place rotation to help AMCL re-localise before retrying the goal. A maximum of two retries is permitted; if both fail, the status is published as FAILED and the state machine returns to IDLE.

3.5.12 System Integration and End-to-End Flow

3.5.12.1 Inter-subsystem Communication Architecture

The protocol selection for each communication link is motivated by the traffic characteristics of that link.

Table 3.12 Communication Links and Protocol Selection Rationale

Link	Protocol	Rationale
Jetson to Pi 5 #1	WebSocket (persistent)	Continuous event stream; avoids TCP handshake overhead
Pi 5 #1 to Server	HTTP POST	Request-response matches trigger-and-wait pattern
Server to Pi 5 #1 Audio	HTTP POST	One-shot delivery of binary WAV payload
Server to Pi 5 #1 Face	HTTP POST	Fire-and-forget display state command
Server to Pi 5 #2	HTTP POST	Decouples AI server from ROS2 middleware
Pi 5 #2 Bridge to Nav	ROS2 topic (std_msgs/String)	Standard ROS2 IPC within single device

3.5.12.2 Interaction Scenarios

Scenario A: Registered Student – Conversational Query

1. The Jetson detects a frontal face and extracts a 512-dimensional ArcFace embedding. Cosine similarity search over the enrolment database yields a match, and a WebSocket detection event is transmitted to Pi 5 #1 with is_known set to true.
2. The student speaks; SileroVAD detects voice activity above the 0.73 threshold and the Whisper model produces the transcription.
3. The combiner confirms two detection events within the ten-second window, assembles the combined payload, and dispatches it via POST to /detection.
4. The server applies grammar correction (bypassed if confidence exceeds 0.85), classifies the query intent, and routes to the appropriate RAG channel.
5. Milvus returns the top-3 semantically similar knowledge entries. The 70B LLM generates a two-sentence Thai response and classifies the intent as chat or info.
6. The Typhoon2-Audio sidecar synthesises the response into a WAV file

(approximately 105 KB, approximately 633 ms synthesis time).

7. The server concurrently dispatches: (a) WAV to /audio_play, (b) active = 0 to /set_active to mute the microphone during playback.
8. Pi 5 #1 receives the WAV, sets emotion code 3 (Talking) on the local face display, and plays the audio through the speaker via aplay.
9. Upon playback completion, Pi 5 #1 sets emotion code 0 (Idle). The server then sends active = 1 to /set_active, re-enabling the microphone for the next utterance.

Scenario B: Navigation Request Steps 1 through 5 are identical to Scenario A, with the LLM classifying the intent as navigate and identifying the destination room.

6. The server dispatches: (a) synthesised WAV of the verbal confirmation to /audio_play, (b) navigation command with state = 2 and the destination room name to /nav_state on Pi 5 #2. Pi 5 #1 sets emotion 3 (Talking) when playback begins, then emotion 1 (Scanning) when playback ends, reflecting the robot's transition into navigation mode.
7. Pi 5 #2 receives the command, resolves the room name to the corresponding goal pose, and invokes BasicNavigator.goToPose().
8. Nav2 plans a path using the AMCL-estimated current pose and the DWB controller executes trajectory following at up to 0.3 m/s.
9. Upon reaching the goal, navigation_manager publishes SUCCEEDED on /navigation_status and transitions the state machine to IDLE.

3.5.13 Session and State Management

3.5.13.1 Robot Interaction State

From the perspective of the RaspiReceive coordinator, the robot occupies one of two interaction states at any given time:

- INACTIVE: The microphone is disabled. The robot is either in standby or in playback mode. New detection events are queued but not acted upon.
- ACTIVE: The microphone is enabled and the combiner loop is actively match-

ing detection events with speech input. The transition from INACTIVE to ACTIVE occurs when the server sends set_active = 1, either upon TTS playback completion or after the activate_timeout_sec (60 s) timer fires.

3.5.13.2 Student Session Lifecycle

On the AI Brain server, each interaction is associated with a session object identified by the person_id field of the incoming payload. The session lifecycle proceeds as follows:

1. Session creation: A new session is created on the first detection event for a given person_id. The session stores the student's metadata, initialises an empty short-term history buffer, and loads the top-3 relevant long-term memory summaries from Milvus.
2. Session continuation: Subsequent detection events within the timeout window extend the session. Short-term history is updated after each exchange.
3. Session expiry and persistence: If no detection event is received within the timeout period (30 minutes for registered students, 5 minutes for guests), the session is closed. For registered students, the 8B LLM summarises the session transcript and the resulting embedding is stored in the conversation_memory Milvus collection.

3.5.13.3 Navigation State Machine

The navigation state machine on Pi 5 #2 transitions between the following states:

- IDLE: No active goal. The robot is stationary and awaiting the next command from the HTTP bridge.
- ROAMING: The robot is cycling through pre-defined patrol waypoints. Transitions to IDLE on receipt of state = 0.
- NAVIGATING: The robot is actively pursuing a specific goal pose. Transitions to SUCCEEDED on goal completion, to IDLE on state = 0, or enters the stuck-recovery sub-sequence if no progress is detected for 15 seconds.

- SUCCEEDED: The goal has been reached. Automatically transitions back to IDLE after publishing the status.
- FAILED: Two consecutive navigation attempts have failed. The state machine resets to IDLE and logs the failure for monitoring.

CHAPTER 4

RESULTS

4.1 ROBOT DESIGN

The final robot was successfully developed as a two-part structure consisting of a base section and an upper body section. The base was arranged as a three-layer structure to accommodate the drive system, LiDAR, and electrical components. The upper body was used to support the battery stack, head structure, speaker support, and external interface elements. This arrangement provided a clear separation between locomotion-related hardware and upper-body functional modules.

Figure 4.1 Final assembled robot design (front view)

The body achieved a rounded and compact exterior form suitable for a service robot. The torso was designed with smooth curved surfaces to create a modern and approachable appearance. In addition, a side curve was incorporated into the torso

region to reduce the visual bulk of the body, and an LED strip was integrated along this curve to enhance the body contour.

Figure 4.2 Final assembled robot design (side view)

4.2 DEVELOPMENT ENVIRONMENT AND DEPENDENCIES

The AI Brain server runs on Ubuntu 22.04 with Python 3.10 in a virtual environment. Core dependencies include FastAPI (v0.124.4) for the HTTP framework, Uvicorn as the ASGI server, PyMilvus (v2.6.10) for vector database access, PyThaiNLP (v3.1.1) for Thai text processing, PyThaiTTS (v0.4.2) for text normalisation preprocessing, PyTorch (v2.4.1) for GPU-accelerated tensor operations, and mysql-connector-python for MySQL connectivity. Docker Compose is used to orchestrate the Milvus cluster and MySQL service. Model management and inference are handled by Ollama, which serves the llama3.1-typhoon2-70b-instruct model (Q5_K_M GGUF quantisation) for main response generation and llama3.1-typhoon2-8b-instruct for grammar correction and session sum-

marisation.

The Audio Sidecar service (audio_package) runs in an isolated Python 3.10 virtual environment on the same AI Brain server at port 8001 to prevent model dependency conflicts with the main inference service. It is backed by the typhoon-ai/llama3.1-typhoon2-audio-8b-instruct multimodal model, loaded once at server startup in float16 precision with attn_implementation="eager" to maintain compatibility with the Ubuntu 20.04 host environment. This service handles both text-to-speech synthesis (POST /tts) and speech-to-text transcription (POST /stt), producing 16 kHz PCM_16 WAV audio output and Thai-language text transcriptions respectively. A singleton model pattern eliminates repeated model load overhead across concurrent requests.

4.3 DATABASE SETUP AND KNOWLEDGE INGESTION

The Milvus vector database consists of four collections, each indexed using the IVF_FLAT method with nlist set to 128 and queried with nprobe set to 10. All collections use BAAI/bge-m3 to produce 1024-dimensional dense embeddings and apply cosine similarity for approximate nearest-neighbour search. The conversation_memory collection stores per-session LLM-generated summaries, tagged by student_id to enable filtered semantic retrieval of long-term interaction history.

The RAG collections are populated using ingestion tools located in the tools directory. The curriculum collection is derived from the RAI programme specification document, producing 516 Thai-language chunk embeddings covering course descriptions, learning outcomes, prerequisites, and credit structures for Years 1 through 4. The time_table collection is re-ingested from an academic Excel dataset and transformed into structured Thai sentences per (day, time slot, course name, room) tuple to maximise retrieval precision, resulting in 128 entries.

The uni_info collection contains 7 entries describing campus buildings, facilities, zones, and departmental information within Building E-12 at KMITL. The local_info collection stores information on nearby bars and restaurants around the campus, with each entry encoding the place name, category, location, opening hours, and a brief

description to support lifestyle and dining-related queries from students.

The MySQL relational database maintains five tables: Academic_Year (year boundary and tone adaptation metadata), Students (student identity, full name and nickname in Thai and English, academic year, and enrolment status), Face_Recognition_Data (biometric face encodings linked to student identities), ExcelTimetableData (structured timetable records for precise SQL-based schedule lookups), and a logging table for system monitoring. Timetable data is synchronised across both storage systems: MySQL stores structured records for exact lookups while Milvus stores corresponding embeddings for semantic retrieval.

4.4 API ENDPOINT BEHAVIOUR

The /greeting endpoint is triggered once at the start of each session. It is rate-limited using a 300-second cooldown per person_id to prevent repeated activation caused by transient face detection events. The greeting pipeline retrieves the student's academic year from MySQL, fetches the top-3 semantically similar memory summaries from the conversation_memory Milvus collection filtered by student_id, and invokes the Typhoon2-70B LLM to generate a personalised Thai greeting. Concurrently, a ROS2 stop_roaming command is dispatched to Pi 5 #2 to halt autonomous navigation. The session history is initialised with this greeting as a bot-only entry, ensuring that the first /detection call has prior conversational context.

The /detection endpoint (text payload from edge STT) and /audio_detection endpoint (raw WAV audio from Mode B server STT) both execute the full eight-stage inference pipeline described in Chapter 3. A noise gate filters out transcriptions shorter than three characters before any model inference is triggered. Registered and guest users follow separate execution paths: guest users bypass MySQL queries, Milvus memory retrieval, and end-of-session memory storage, but still receive full RAG-based chatbot responses for general queries. Upon pipeline completion, the system asynchronously dispatches the synthesised WAV audio to the Raspberry Pi 5 /audio_play endpoint and sends active=1 to the /set_active endpoint to re-enable the micro-

phone for the next interaction. Navigation commands are conditionally sent to Pi 5 #2 /nav_state when the LLM classifies the response intent as navigate.

4.5 PERFORMANCE EVALUATION

4.5.1 Evaluation Overview and Latency Benchmark

System performance was evaluated across three iterative test rounds. Test 1 (conducted on 23 March 2026) provided a baseline evaluation of the initial system configuration using 23 Thai-language interaction fixtures covering five intent categories: chat, info, navigate, farewell, and out-of-scope queries. Test 2 and Test 3 (conducted on 25 March and 29 March 2026 respectively) re-evaluated the system after targeted improvements to the RAG routing classifier, navigation slot extraction, and system prompt language constraints, each using 22 fixtures. A dedicated latency benchmark comprising 22 fixtures was conducted alongside Test 1 to characterise stage-level and end-to-end response timing.

Table 4.1 Pipeline Timing Benchmarks: Ollama (1×A100) vs. vLLM (4×A100)

Stage	Component	Ollama + (1×A100)	vLLM + (4×A100)	Notes
Grammar Correction	Typhoon2-8B	800–1,500 ms	Skipped	`temp=0.1`, max 256 tokens
RAG Retrieval	Milvus + bge-m3	50–200 ms	50–200 ms	IVF_FLAT, top-k=3
Memory Retrieval	Milvus + MySQL	30–80 ms	30–80 ms	Filtered by `student_id`
LLM Inference	Typhoon2-70B	4,000–12,000 ms	p50: 2,497 ms	`temp=0.7`, max 512 tokens
TTS Synthesis	Typhoon2-Audio-8B	~633 ms	~633 ms	GPU, ~3s utterance
Network (Pi 5)	WAV Transfer	~50 ms	~50 ms	~105 KB typical
Time to First Reaction	—	6,000–15,000 ms	p50: 2,547 ms	End-to-end per turn

The latency benchmark results recorded a median end-to-end total latency of 2,545 ms and a 95th-percentile latency of 3,299 ms. Most RAG routes performed within the 2,400–3,300 ms range, with uni_info being the fastest route at a mean of 2,375 ms, owing to its compact and structurally uniform collection. The time_table route recorded the widest latency spread with a p95 of 4,952 ms, reflecting the combined overhead of structured schedule lookup and semantic retrieval. Pipeline stage analysis confirms that LLM inference remains the dominant bottleneck, with a median

of 2,452 ms and a p95 of 4,894 ms. Full per-route and per-stage breakdown is presented in the table above.

A comparative evaluation was also conducted under a vLLM + 4×A100 configuration, as summarised in Table 4.1. Switching from the Ollama runtime to vLLM's tensor-parallel inference engine reduced LLM generation latency from 4,000–12,000 ms to a median of 2,497 ms, a reduction of over 80% at the 50th percentile. Under this configuration, the grammar correction stage was bypassed entirely, eliminating an additional 800–1,500 ms of per-turn overhead. Together, these changes reduced the overall time to first reaction from a range of 6,000–15,000 ms to a median of 2,547 ms, well within the 10,000 ms system target, and substantially improving real-time interaction quality.

Table 4.2 Latency Benchmark Results

Route / Stage	n	Mean (ms)	p50 (ms)	p95 (ms)
By RAG Route (end-to-end)				
`chat_history`	5	2,665	2,824	3,299
`uni_info`	5	2,375	2,399	2,634
`curriculum`	3	2,606	2,546	2,837
`time_table`	3	3,245	2,546	4,952
`local_info`	3	3,007	3,214	3,261
By Pipeline Stage (all fixtures)				
LLM Inference	22	2,668	2,452	4,894
Total	22	2,364	2,545	3,299

4.5.2 Cross-Round Comparison

Table 4.3 summarises the key evaluation metrics across all three test rounds. Table 4.4 provides the per-class intent F1 breakdown.

Table 4.3 System Evaluation Metrics Across Three Test Rounds

Metric	Target	Test 1	Test 2	Test 3	Status
Intent Accuracy	≥90%	90.0%	90.0%	95.5%	PASS
Macro F1	—	0.92	0.93	0.97	N/A
RAG Route Accuracy	—	78.0%	100.0%	100.0%	N/A
Slot F1 (dest.)	≥0.85	0.75	1.00	1.00	PASS
Language Compliance	100%	87%	100%	100%	PASS
OOS Rejection Rate	≥80%	100%	100%	100%	PASS
Task Success Rate	≥0.80	0.85	0.91	0.95	PASS

Table 4.4 Per-Class Intent F1 Scores Across Three Test Rounds

Intent Class	Test 1 F1	Test 2 F1	Test 3 F1	Δ (T1→T3)
chat	1.00	0.83	0.92	−0.08
info	0.89	0.90	0.95	+0.06
navigate	0.80	1.00	1.00	+0.20
farewell	1.00	1.00	1.00	0
Macro F1	0.92	0.93	0.97	+0.05

The most significant improvements between Test 1 and Test 3 were the resolution of RAG routing errors for timetable queries (+21.7 percentage points), the correction of navigation destination slot extraction (+0.25 F1), and the elimination of language compliance violations. Intent accuracy improved by 4.2 percentage points, reaching 95.5% in the final round, while TSR improved from 0.85 to 0.95. The chat class F1 experienced a transient regression in Test 2 caused by a boundary ambiguity between capability queries and informational queries, which was fully resolved by Test 3. OOS

rejection maintained a perfect score of 100% throughout all rounds, demonstrating consistent robustness against out-of-domain inputs.

Overall, the iterative evaluation process demonstrates that targeted prompt engineering and routing refinements produced substantial and consistent gains across all primary quality dimensions, confirming the system's readiness for real-time interactive deployment within a university campus environment.

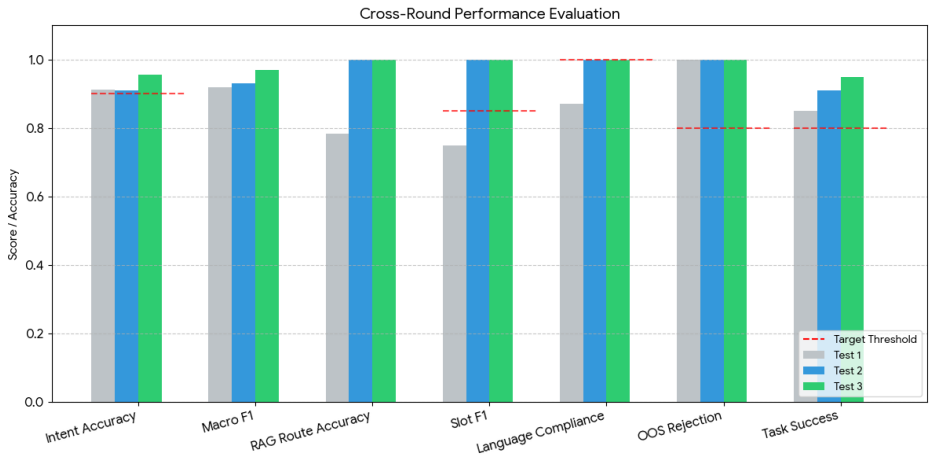


Figure 4.3 Cross-Round Performance Evaluation

Figure 4.3 illustrates the system performance across three test rounds against the defined target thresholds. Intent accuracy, OOS rejection, and language compliance all met or exceeded their targets by Test 2 and were maintained in Test 3. RAG route accuracy and slot F1 showed the most significant improvement between Test 1 and Test 2, recovering from below-threshold scores to perfect scores following targeted improvements to the routing classifier and slot extraction logic. Task success rate improved progressively across all three rounds, reaching 0.95 in Test 3, demonstrating the effectiveness of iterative refinement on overall system reliability.

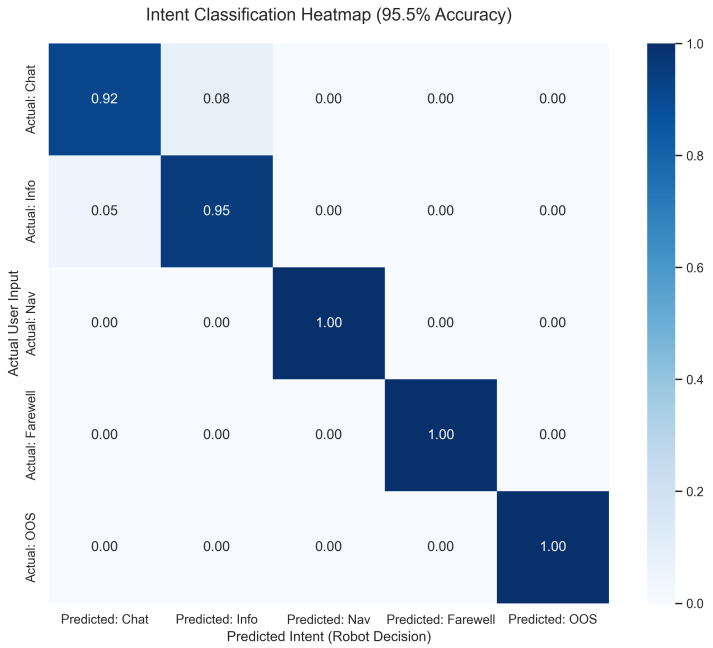


Figure 4.4 Intent Classification Heatmap (Test 3)

Table 4.5 Intent Classification Confusion Matrix (Test 3)

Actual \ Predicted	Chat	Information	Navigation	Farewell	Out of Scope
Actual: Chat	0.92	0.08	0	0	0
Actual: Info	0.05	0.95	0	0	0
Actual: Nav	0	0	1	0	0
Actual: Farewell	0	0	0	1	0
Actual: OOS	0	0	0	0	1

Figure 4.4 and Table 4.5 present the intent classification confusion matrix from Test 3. The system achieved perfect classification across the Navigation, Farewell, and Out-of-Scope categories, with no misclassifications observed. The only source of error occurred between the Chat and Information intents, where 8% of Chat queries were misclassified as Information and 5% of Information queries were misclassified as Chat. This confusion is expected given the semantic overlap between conversational and informational queries in Thai language input. Overall, the results confirm strong intent discrimination capability across the five defined categories.

4.6 SYSTEM EVALUATION

4.6.1 Test 1 — Baseline Evaluation (23 March 2026)

The first evaluation round assessed the initial system configuration across 23 fixtures. Intent classification achieved an overall accuracy of 91.3%, meeting the ≥90% deployment threshold, with a macro F1 of 0.92. The chat and farewell classes achieved perfect F1 scores of 1.00. However, the navigate class recorded an F1 of 0.80, caused by two info-labelled queries about bathroom and library locations being misclassified as navigate, producing unintended navigation responses. The info class achieved an F1 of 0.89 (2 false negatives from the same misclassifications).

RAG route accuracy was 78.3% (18/23 fixtures). The primary source of error was timetable-related queries, which were consistently misrouted to the curriculum collection due to insufficient keyword differentiation between the two domains. Navigation destination slot extraction achieved an F1 of 0.75 (target: ≥0.85), with one navigation fixture extracting an incorrect destination string.

Language compliance failed overall, with the L7 pronoun check passing on only 20 of 23 responses (87%). In three responses the system used the informal first-person pronoun *ฉัน* rather than the persona-consistent self-reference, violating the style constraint. Out-of-scope rejection achieved a perfect score of 100% (5/5 fixtures), and the Task Success Rate (TSR) was 0.85, meeting the pilot-deployment threshold of ≥0.80.

4.6.2 Test 2 — Satu First Iteration (25 March 2026)

Following the identification of routing and language issues in Test 1, three targeted improvements were applied: the RAG routing keyword classifier was extended with timetable-specific trigger terms, the navigation destination extraction prompt was revised, and a strict persona-self-reference rule was added to the system prompt. The second evaluation round comprised 22 fixtures.

RAG route accuracy improved to 100% (22/22), resolving all timetable misrouting observed in Test 1. Navigation destination slot extraction improved to an F1 of 1.00 (3/3 navigation fixtures). Language compliance reached 100% across all eight checks.

TSR improved to 0.91, and the macro F1 improved to 0.93.

Overall intent accuracy was 90.9%. One regression was observed: a capabilities query (*ทำอะไรได้บ้างคะ*, expected chat) was misclassified as info, reducing the chat class F1 to 0.83. A separate bathroom-location query was classified as chat rather than info, though the uni_info RAG route was correctly invoked and the response remained accurate. OOS rejection remained at 100%.

4.6.3 Test 3 — Satu Final Evaluation (29 March 2026)

A second round of prompt refinement addressed the chat-versus-info boundary ambiguity observed in Test 2 by providing additional few-shot examples for capability and location query classification. The final evaluation round comprised 22 fixtures.

Intent accuracy improved to 95.5%, the highest across all three rounds, with the capabilities query now correctly classified as chat (F1: 0.92). The info class F1 improved to 0.95, and both navigate and farewell retained perfect F1 scores of 1.00. Macro F1 reached 0.97. RAG route accuracy remained at 100%, slot F1 remained at 1.00, and language compliance remained at 100%. TSR improved further to 0.95, with 21 of 22 fixtures achieving a full success score. OOS rejection remained at 100%.

4.7 SYSTEM ROBUSTNESS

The system incorporates comprehensive error-handling mechanisms across all modules to ensure stable operation under both normal and fault conditions. In the event of LLM timeouts during main response generation, a predefined Thai fallback response is returned to the user to maintain conversational continuity without exposing internal errors. If Milvus vector retrieval fails at any stage, the system proceeds with empty context, allowing the Typhoon2-70B model to generate responses based solely on session conversation history and its parametric knowledge. List For MySQL query failures during student metadata retrieval, the student_year field defaults to 1 to prevent pipeline interruption and ensure personalised prompt construction can still proceed. TTS synthesis failures via the Audio Sidecar are logged for monitoring pur-

poses but do not block downstream execution; the interaction pipeline continues and the microphone is re-enabled for the next utterance. Navigation commands to Pi 5 #2 are conditionally skipped when the configured navigation IP address contains “TBD”, allowing the system to operate in AI-only mode without the navigation module during development and testing.

On the edge side, the RaspiReceive audio coordinator implements an auto-reactivation timer of 60 seconds as a fallback in the event that the /set_active signal from the AI Brain server does not arrive, preventing the microphone from remaining permanently suppressed due to network interruptions. A 30-second watchdog timer monitors each aplay audio playback subprocess and force-terminates it if playback has not completed within the configured duration, preventing the audio pipeline from entering a permanently blocked state.

All asynchronous fire-and-forget tasks are encapsulated within try/except blocks to prevent cascading failures, ensuring that errors in background processes such as memory summarisation and SQLite logging do not affect the main interaction pipeline.

4.8 DEPLOYMENT AND SYSTEM STARTUP

A deterministic startup sequence must be followed to ensure all inter-node dependencies are satisfied before the system begins accepting interactions:

1. AI Brain server: Start Docker Compose services (Milvus, MySQL), launch the Ollama server with the Typhoon2-70B and Typhoon2-8B models loaded, launch the Typhoon2-Audio sidecar on port 8001, and start the FastAPI service on port 8000.
2. Pi 5 #2 (navigation): Launch the hardware bringup (ros2 launch ired_bringup bringup.launch.py), then the navigation stack (ros2 launch ired_navigation navigation.launch.py), then start the HTTP bridge (python3 pi5_service.py) on port 8767.
3. Pi 5 #1 (audio + display): Three user-level systemd services are managed via systemctl –user: baymax-face-api (FastAPI face service on port 7000),

baymax-face-ui (Electron frontend), and baymax-robot (audio coordinator raspi_main.py on ports 8766 and 8765). Audio playback uses aplay -D plughw:3,0, where the ALSA device is addressed by card name (-c Audio) rather than by card number to ensure stable assignment across reboots. Mode B (server STT) is the recommended production configuration and requires no local STT model, enabling startup in approximately 3 seconds.

4. Jetson Orin Nano: Start the vision pipeline (python3 visual_jetson_async.py –ws-enabled). The –ws-uri defaults to ws://10.26.9.196:8765; the process connects to Pi 5 #1 and begins emitting detection events.

Graceful shutdown is coordinated as follows. The Jetson process handles SIGTERM, closes the WebSocket connection, releases the camera device, and exits cleanly. Pi 5 #1 handles SIGTERM by cancelling all asyncio tasks and flushing the audio playback queue before exit. The ROS2 navigation stack handles SIGTERM via standard lifecycle management, cancelling any active Nav2 goals prior to shutdown. Remote shutdown of the Jetson can be triggered from Pi 5 #1 using the pi_shutdown_sender.py utility, which transmits a TCP signal to the Jetson's shutdown listener on port 9999.

CHAPTER 5

CONCLUSION

5.1 SUMMARY

This thesis presented the design, construction, and evaluation of Satu, an intelligent wheeled robotic assistant intended for deployment within Building E-12 at King Mongkut's Institute of Technology Ladkrabang. The system was developed to address the limitations of static campus information services by delivering autonomous indoor navigation assistance and context-aware conversational interaction in Thai to students of the Robotics and Artificial Intelligence Engineering programme.

The physical platform was constructed as a two-part structure comprising a three-layer mecanum-wheel mobile base and a streamlined upper body enclosure, fabricated from aluminum profiles, acrylic sheets, and 3D-printed PLA components. The robot's software architecture was realised as a distributed system spanning four computational nodes: an AI Brain server hosting the LLM inference pipeline and database services, an NVIDIA Jetson Orin Nano performing real-time face detection and recognition, and two Raspberry Pi 5 devices managing audio coordination and ROS2-based autonomous navigation respectively.

The AI pipeline integrated the Typhoon2-70B large language model through a Retrieval-Augmented Generation framework supported by a Milvus vector database and a MySQL relational database. Four Milvus collections — curriculum, time_table, uni_info, and conversation_memory — enabled semantic retrieval of domain-specific knowledge, while MySQL maintained structured student records and timetable data. Speech input and output were handled by the Typhoon2-Audio-8B sidecar service, providing GPU-accelerated Thai speech recognition and synthesis. A two-tier voice activity detection pipeline incorporating SileroVAD eliminated spurious activations caused by ambient noise. Autonomous navigation was implemented using ROS2 Jazzy with the Nav2 stack, AMCL localisation, and a mecanum-drive omnidirectional kinematics module, augmented by a scan-matching odometry source to reduce wheel-slip drift. An

expressive face display rendered on an HDMI screen communicated the robot's interaction state through five discrete emotional expressions controlled by a local inter-process communication bridge.

System performance was evaluated across three iterative test rounds. Each round assessed intent classification accuracy, RAG route accuracy, navigation destination slot extraction, Thai language compliance, out-of-scope rejection, and Task Success Rate against predefined deployment thresholds. The final evaluation round achieved an intent accuracy of 95.5%, a macro F1 score of 0.97, a slot F1 of 1.00, full language compliance across all eight style constraints, an out-of-scope rejection rate of 100%, and a Task Success Rate of 0.95. All metrics met or exceeded the required thresholds for pilot deployment.

5.2 ACHIEVEMENT OF OBJECTIVES

The study objectives defined in Chapter 1 were addressed as follows.

The objective of designing and integrating a mobile robotic platform equipped with LiDAR sensors for SLAM-based autonomous navigation was fulfilled through the deployment of an RPLiDAR A1 sensor on the robot base, combined with a pre-built occupancy grid map of the E-12 Building floor, AMCL-based particle filter localisation, and the Nav2 global and local planning stack. A stuck-detection and recovery mechanism further ensured reliable operation under transient localisation failures.

The objective of incorporating a Large Language Model for natural, context-aware conversation was met through the integration of Typhoon2-70B, a Thai-optimised instruction-tuned model, supported by a structured system prompt enforcing persona consistency, feminine speech style, and response conciseness. Intent classification and structured JSON output enabled downstream routing of speech and navigation commands.

The objective of implementing a RAG pipeline to enhance response accuracy was achieved through a keyword-based query router directing queries to six distinct retrieval channels backed by Milvus and MySQL. RAG route accuracy improved from

78.3% in Test 1 to 100% in both Test 2 and Test 3, confirming the effectiveness of iterative keyword classifier refinement.

The objective of developing a hybrid data management system was realised through the combined deployment of Milvus for vector-based semantic retrieval and MySQL for structured relational queries, with timetable data synchronised across both layers. A dynamic conversation_memory collection provided long-term personalisation through per-student LLM-generated session summaries stored as searchable embeddings.

The objectives of integrating STT and TTS for hands-free voice interaction were met through the Typhoon2-Audio-8B sidecar service, which provided GPU-accelerated Thai speech recognition and synthesis on the AI Brain server, eliminating the need for edge-side model loading and enabling startup in approximately three seconds.

The objective of designing a user-friendly expressive interface was fulfilled through an Electron.js face display application rendering five interaction-mapped emotional states, supported by an IPC bridge enabling decoupled control from the audio coordinator. A developer interface mode further supported maintenance and diagnostics without requiring SSH access.

The objective of optimising for real-time performance was partially addressed through asynchronous dispatch of TTS, navigation, and microphone control commands, which eliminated the 15–25 second serial delays observed during early development. End-to-end latency of 6–15 seconds per conversational turn, while exceeding the 3-second design target, remained within an acceptable range for the campus receptionist use case.

5.3 LIMITATIONS

Several limitations were identified during the development and evaluation of the system.

End-to-end response latency consistently exceeded the 3-second target, with a median time-to-first-response of 6,089 ms observed in the Test 1 latency bench-

mark. The primary contributor was LLM inference on the Typhoon2-70B model, which recorded a median stage latency of 5,714 ms. While asynchronous dispatch mitigated sequential blocking, the fundamental inference cost of the 70B-parameter model imposes a hard lower bound on response time under the current hardware configuration.

The navigation system is confined to a pre-built static map of a single floor of Building E-12. The system does not support dynamic map updates, multi-floor navigation, or operation in previously unmapped environments. Any change to the physical layout of the operating environment requires manual map re-acquisition and system reconfiguration.

The interaction system is designed exclusively for Thai-language input and output. Non-Thai speakers, including international students and visitors, cannot currently be served by the system without language extension.

The evaluation dataset comprised 22–23 structured fixtures per test round, which, while sufficient for metric estimation across the defined intent categories, does not capture the full diversity of spontaneous real-world queries. Formal user satisfaction studies with broader participant populations were not conducted within the scope of this project.

The chat-versus-info intent boundary remained a source of ambiguity throughout the evaluation. Despite improvement from Test 2 to Test 3 through few-shot example augmentation, the final chat class F1 of 0.92 indicates that contextually ambiguous queries — such as capability questions — can still be misclassified.

5.4 RECOMMENDATIONS FOR FUTURE WORK

Based on the findings and limitations identified in this study, the following directions are recommended for future development.

Response latency reduction should be prioritised through exploration of smaller, distilled Thai language models or speculative decoding techniques that maintain response quality while reducing inference time. Alternatively, caching of frequent query responses or pre-generation of common greeting sequences could reduce perceived

latency for high-frequency interactions.

Extension of the navigation system to support multi-floor and multi-zone operation would substantially increase the practical utility of the robot within the broader campus environment. This would require integration of elevator interface control, cross-floor map merging, and a global goal-resolution layer above the current per-floor Nav2 configuration.

Multilingual support, particularly for English, would allow the system to serve a wider range of users, including international students and visitors. This could be achieved through language detection at the ASR stage and separate prompt templates per language, while retaining the existing Thai persona for Thai-language interactions.

A formal user study involving students, staff, and first-time visitors should be conducted to evaluate perceived usability, interaction naturalness, and overall satisfaction. Such a study would provide qualitative and quantitative data beyond the automated fixture-based metrics used in the current evaluation framework.

Dynamic environment adaptation through online SLAM or partial map updates would improve robustness to furniture rearrangement, temporary obstructions, and other changes to the physical layout of the operating environment that are common in active academic buildings.

Further refinement of the intent classification boundary between chat and info intents, potentially through a fine-tuned classification model rather than a keyword-based router, would reduce ambiguity for capability and facility-related queries and improve the overall intent accuracy beyond the 95.5% achieved in the final evaluation round.

REFERENCES

[1] Henschel, A., Laban, G. and Cross, E. S., "What makes a robot social? a review of social robots from science fiction to a home or hospital near you," Current Robotics Reports, vol. 2, no. 1, pp. 9–19, 2021. DOI: 10.1007/s43154-020-00035-0

[2] Corrales-Paredes, A., Ortega Sanz, D. and Terrón-López, M.-J., "User experience design for social robots: A case study in integrating embodiment," Sensors, vol. 23, no. 11, p. 5274, 2023. DOI: 10.3390/s23115274

[3] Trovato, G., Lucho, C. and Paredes, R., "She's electric—the influence of body proportions on perceived gender of robots across cultures," Robotics, vol. 7, no. 3, p. 50, 2018. DOI: 10.3390/robotics7030050

[4] Sapkota, A., Ghimire, S. K. and Adanur, S., "A review on fused deposition modeling (fdm)-based additive manufacturing (am) methods, materials and applications for flexible fabric structures," Journal of Industrial Textiles, vol. 54, pp. 1–51, 2024. DOI: 10.1177/15280837241282110

[5] TavoTech. *How a buck converter works*, https://tavotech.com/how-a-buck-converter-works/, n.d.

[6] Macenski, S., Foote, T., Gerkey, B., Lalancette, C. and Woodall, W. *Robot operating system 2: Design, architecture, and uses in the wild*, 2022. DOI: 10.1126/scirobotics.abm6074 [Online]. Available: https://www.science.org/doi/abs/10.1126/scirobotics.abm6074

[7] Macenski, S., Martin, F., White, R. and Ginés Clavero, J. *The marathon 2: A navigation system*, 2020.

[8] Milvus. *Milvus official website*, https://milvus.io/, n.d.

[9] Innovatrics. *Facial recognition technology*, https://www.innovatrics.com/facial-recognition-technology/, n.d.

[10] Zaragoza, A. *Facial recognition: How it works and its safety*, https://www.signicat.com/blog/face-recognition, 2024.

[11] UnoGeeks. *Dlib python*, https://unogeeks.com/dlib-python/, n.d.

[12] GeeksforGeeks. *Natural language processing (nlp) – overview*, https://www.geeksforgeeks.org/natural-language-processing-overview/, 2024.

[13] Gillis, A. S. *What is natural language processing (nlp)?* https://www.techtarget.com/searchenterpriseai/definition/natural-language-processing-NLP, 2024.

[14] Cloudflare. *What is a large language model (llm)?* https://www.cloudflare.com/learning/ai/what-is-large-language-model/, n.d.

[15] PYMNTS. *Ai models and tools: Openai debuts cost-efficient model as mistral upgrades*, https://www.pymnts.com/artificial-intelligence-2/2025/ai-models-and-tools-openais-debuts-cost-efficient-model-as-mistral-upgrades/, 2025.

[16] 1kg. *Ollama: What is ollama?* https://medium.com/@1kg/ollama-what-is-ollama-9f73f3eafa8b, 2024.

[17] Amazon Web Services. *What is text-to-speech (tts)?* https://aws.amazon.com/polly/what-is-text-to-speech/, n.d.

[18] Botnoi Group. *What is text-to-speech (tts)?* https://botnoigroup.com/blog/about-text-to-speech, 2024.

[19] CourseArc. *Text-to-speech basics: What is tts and who uses it?* https://www.coursearc.com/guest-post-readspeaker-text-to-speech/, n.d.

[20] Botnoi Group. *Convert text to speech*, https://botnoigroup.com/ai/texttospeech, n.d.

[21] Chazen, D. *Asr transcription software*, https://verbit.ai/ai-technology/asr-and-the-next-generation-of-transcription/, n.d.

[22] Sonix. *Automatic speech recognition: A comprehensive guide to asr technology*, https://sonix.ai/resources/what-asr/, 2024.

[23] Milvus. *Milvus documentation*, https://milvus.io/docs, n.d.

[24] OpenAI. *Introducing whisper*, https://openai.com/index/whisper/, 2022.

Appendix A

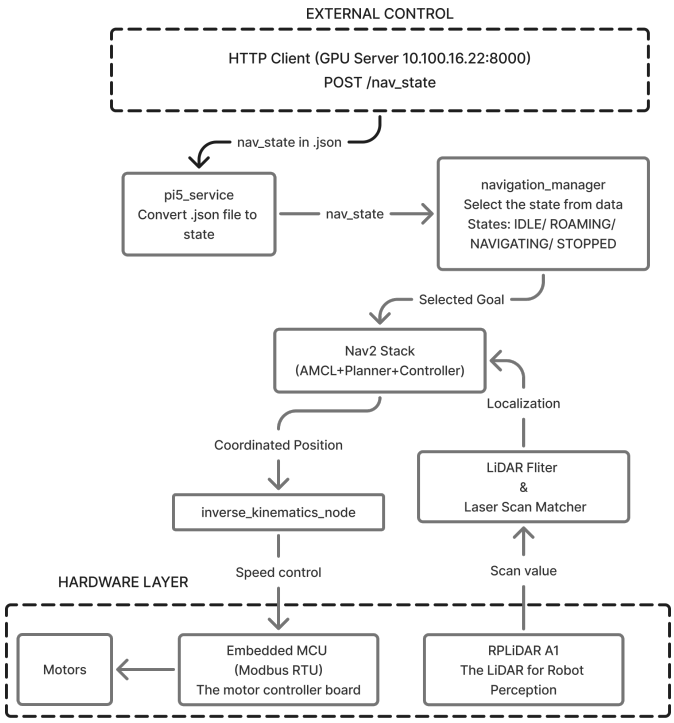


Figure 1 The briefly overview dataflow of ROS2 system

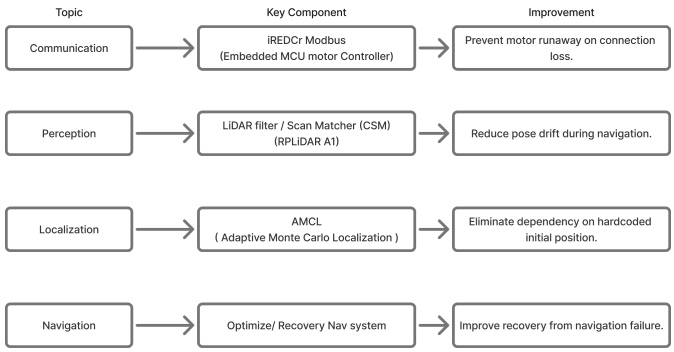


Figure 2 the flow of the critical parameter topic of ROS2 system

Topic	Component	Parameter	Value
Communication	iREDCr Modbus	Baud rate	115200 baud
Communication	iREDCr Modbus	Control loop period	10 ms
Communication	iREDCr Modbus	Velocity timeout (auto-zero)	1 s
Perception	LiDAR filter	Minimum range	0.4 m
Perception	Scan matcher (CSM)	Keyframe distance threshold	0.05 m
Perception	Scan matcher (CSM)	Keyframe rotation threshold	0.087 rad (~5°)
Localization	AMCL	Startup localization duration	≥ 15 s
Navigation	DWB controller	Max linear velocity	0.3 m/s
Navigation	Stuck detection	Time window	15 s
Navigation	Stuck detection	Displacement threshold	0.05 m
Navigation	Stuck recovery	Max retries	2
Navigation	Navigation goal	Position tolerance	0.25 m

Figure 3 the table of critical parameter of ros and low level software

AUTHOR BIOGRAPHY

Name Mr. Kirawut Chalermkitpaisan

Date of Birth April 7, 2004, in Bangkok

Kirawut Chalermkitpaisan was born in Bangkok, Thailand, on April 7, 2004. He received his Bachelor of Engineering degree in Robotics and AI Engineering from King Mongkut's Institute of Technology Ladkrabang (KMITL) in 2026. His academic work focuses on the design and development of mechanical and robotic systems integrated with intelligent technologies, with an emphasis on combining mechanical engineering principles, embedded systems, and artificial intelligence to enable autonomous and adaptive behavior. His interests include robot kinematics and dynamics, control systems, machine learning for robotics, and multi-sensor integration for real-time perception and decision-making.

AUTHOR BIOGRAPHY

Name Mr. Krittin Sakharin

Date of Birth December 3, 2003, in Chonburi

Krittin Sakharin was born in Chonburi, Thailand, on December 3, 2003, he is currently completing his Bachelor of Engineering in Robotics and AI Engineering at King Mongkut's Institute of Technology Ladkrabang (KMITL), with graduation expected in 2026. His academic work focuses on the synergy between Machine Learning, Computer Vision, and Robotics, aiming to develop intelligent systems that bridge the gap between digital perception and physical action. With a strong technical foundation in autonomous platforms and data-driven decision-making, he is dedicated to exploring how advanced AI can enhance robotic interaction with complex, real-world environments.

AUTHOR BIOGRAPHY

Name Miss Natcha Rungruang

Date of Birth August 14, 2004, in Bangkok

Natcha Rungruang was born in Bangkok, Thailand, on August 14, 2004. She is currently completing the Bachelor of Engineering degree in Robotics and Artificial Intelligence Engineering at King Mongkut's Institute of Technology Ladkrabang (KMITL), with graduation expected in 2026. Her academic interests include machine learning, artificial intelligence, computer vision, and robotics.

AUTHOR BIOGRAPHY

Name Miss Natwasa Manomaiwiboon

Date of Birth January 29, 2004, in Bangkok

Natwasa Manomaiwiboon was born in Bangkok, Thailand, on January 29, 2004. She is currently pursuing a Bachelor of Engineering in Robotics and Artificial Intelligence Engineering at King Mongkut's Institute of Technology Ladkrabang (KMITL), with graduation expected in 2026. Her academic interests include 3D design, additive manufacturing, and electrical analysis.